

MBS-fast

Подробное руководство по физике, математике, численным методам и запуску

Актуальная PDF-версия для ветки `main`

15 июля 2026

Аннотация

MBS-fast считает рассеяние света несферическими ледяными частицами методом геометрической трассировки лучей с последующим интегралом Physical Optics (PO) по апертурам выходных пучков. Этот документ является самостоятельным PDF-руководством: он не получается из Markdown и предназначен для нормального чтения в PDF. В нем описаны координатные системы, трассировка, коэффициенты Френеля, матрицы Jones и Mueller, повороты базисов, дифракционный edge-интеграл, усреднение по ориентациям, правила выбора сеток, GPU/CPU ускорение, multi-GPU режимы, входные данные, выходные файлы и все основные флаги запуска.

Содержание

1	Назначение программы	5
1.1	Какая физика заложена	5
1.2	Обозначения	6
2	Координатные системы	7
2.1	Лабораторная система	7
2.2	Система частицы и повороты Эйлера	7
2.3	Система грани и апертуры	8
2.4	Система рассеяния	8
3	Трассировка лучей	8
3.1	Пересечение луча с гранью	8
3.2	Пучок как полигон	9
3.3	Отражение и преломление	9

3.4	Оптический путь	10
3.5	Ограничение глубины	10
4	Коэффициенты Френеля	10
4.1	Вертикальная и горизонтальная поляризация	10
4.2	Обычное отражение и прохождение	10
4.3	Нормальное падение	11
4.4	Полное внутреннее отражение	11
4.5	Старый знак Fresnel	12
5	Jones-матрицы и диады	12
5.1	Матрица пучка	12
5.2	Фазовый множитель	12
5.3	Поворот Jones в базис рассеяния	12
5.4	Полный Jones-вклад пучка	13
5.5	Karczewski-ветка	13
6	Преобразование Jones в Mueller	14
6.1	Полная формула из кода	14
6.2	Когерентная и некогерентная сумма	14
6.3	Повороты Mueller	15
7	Дифракция Physical Optics	15
7.1	Интеграл по апертуре	15
7.2	Переход к ребрам полигона	15
7.3	Почему важны точки 0 и 180 градусов	17
7.4	Поглощение	17
8	Clipping невыпуклых частиц и агрегатов	17
8.1	Зачем нужен clipping	17
8.2	Агрегаты	18
8.3	Projected-AABB prefilter	19
9	Размер частицы и масштабирование	19
9.1	Built-in и file particles	19
9.2	Native particle format и экспорт геометрии	20
9.3	Equivalent radius и k_{eq}	22
10	Ориентационное усреднение	23
10.1	Интеграл по ориентациям	23
10.2	Регулярная beta/gamma сетка	23
10.3	Флаг --pole	24
10.4	Oldauto	24

10.5	Sobol, Hammersley, lattice, Monte Carlo	24
10.6	Adaptive и формула выбора числа ориентаций	25
10.7	Файл настроек адаптивной сходимости	26
10.8	Auto, autofull, oldautofull	26
10.9	Почему симметрия столбика не делит Nphi на шесть	27
11	Scattering grid	28
11.1	Theta rows и solid angle	28
11.2	Azimuthal grid	28
11.3	Auto theta grid	28
12	CPU-реализация и AVX	28
12.1	Что параллелится на CPU	28
12.2	AVX-512 packing	29
12.3	Почему важны флаги компиляции	29
13	GPU-реализация	30
13.1	Что ускоряет GPU	30
13.2	Atomic accumulation	31
13.3	Почему fused path быстрее	31
13.4	Multi-GPU	31
14	Сборка	33
14.1	CPU	33
14.2	GPU	33
15	Запуск и диагностика	33
15.1	Минимальный production-пример	33
15.2	CPU из GPU binary	34
15.3	Проверка точек 0 и 180 градусов	34
16	Краткий справочник флагов	34
16.1	Метод и частица	34
16.2	Ориентации	35
16.3	Сетка, ускорение, cutoffs	36
16.4	Output, диагностика, legacy	37
17	Выходные файлы	38
18	Практические правила валидации	40
19	Расширенные математические комментарии	41
19.1	Нормировка интенсивности	41
19.2	Устойчивый предел edge-интеграла	41

19.3	Предел Jones-базиса на полюсах	42
19.4	Симметрия forward/backward Mueller	42
19.5	Ошибка ориентационного усреднения	43
19.6	Практическая формула выбора N_{ori}	43
19.7	Ошибка theta grid	43
19.8	Сходимость по глубине трассировки	44
19.9	Beam cutoffs	44
19.9.1	Единые профили	45
19.9.2	Отдельные пороги	45
19.10	Double против float	46
19.11	FFT/interpolation по phi	47
19.12	Multi-size reuse	48
19.13	Почему таблицы теперь не вылезают	48
19.14	Протокол поиска расхождения CPU/GPU	48
19.15	Протокол выбора production-параметров	49
19.16	Формат частицы из файла	49
19.17	Площадь, объем и центр	50
19.18	Разбор строки output	50
19.19	Матрица validation-тестов	51
19.20	Типичные причины больших расхождений	51
19.21	Контроль conservation-like величин	52
19.22	Минимальные данные для воспроизводимого отчета	52
19.23	English technical summary	53
A	Краткая английская шпаргалка	53
B	Связь с файлами кода	54
C	Приложение: контрольные формулы для отчетов	55
C.1	Относительная ошибка элемента Mueller	55
C.2	Интегральная ошибка по углам	55
C.3	Сравнение времени	55
C.4	Оценка памяти GPU	56
D	Приложение: чеклист перед публикацией чисел	57
D.1	Минимальная таблица в статье или отчете	57
D.2	Что не надо смешивать в одном сравнении	57
E	Appendix: English flag groups	59

1. Назначение программы

MBS-fast решает задачу дальнего поля для частицы с плоскими гранями. Программа не дискретизирует весь объем частицы, как DDA, а строит множество геометрических пучков, которые возникают после отражений и преломлений на гранях. Каждый пучок несет полигон апертуры, направление распространения, оптический путь и комплексную матрицу Jones. После трассировки для каждого выходного пучка считается РО-дифракция по его апертуре, затем вклады пучков суммируются и переводятся в матрицу Mueller.



CPU prepares rays and beams; CPU or GPU evaluates diffraction on the detector grid

Рис. 1: Основной pipeline: геометрия частицы, трассировка, Fresnel/Jones, РО-дифракция, когерентная сумма, Mueller и усреднение ориентаций.

1.1. Какая физика заложена

В расчете используются четыре уровня приближения.

Таблица 1: Физические уровни модели MBS-fast.

Уровень	Что считается	Где появляется в коде
Геометрическая оптика	Лучи отражаются и преломляются на плоских гранях по законам Снеллиуса; слабые ветви отбрасываются cutoffs.	src/scattering, src/tracer, src/Splitting.cpp.

Уровень	Что считается	Где появляется в коде
Fresnel/Jones	Поляризационная амплитуда каждой ветви хранится как комплексная матрица Jones.	Beam::J, Beam::MultiplyJonesMatrix, Splitting.cpp.
Physical Optics	Выходной пучок дает дальнее поле как интеграл по полигональной апертуре.	Handler::DiffractIncline*, HandlerPO::ApplyDiffraction*, CUDA kernels.
Ориентационное усреднение	Для случайной ориентации считается взвешенная сумма Mueller-матриц по сетке Эйлера или квазислучайному набору.	HandlerPOTotal, TracerPOTotal, блоки --oldauto, --adaptive, --auto.

Главная численная особенность: в РО-режиме пучки внутри одной ориентации суммируются когерентно на уровне Jones-амплитуд. Только после суммы Jones матрица переводится в Mueller. Флаг `--incoh` меняет это поведение и делает сумму Mueller по отдельным пучкам.

1.2. Обозначения

Таблица 2: Базовые обозначения, используемые дальше.

Символ	Смысл
λ	Длина волны в микрометрах, задается флагом <code>-w</code> .
$k = 2\pi/\lambda$	Волновое число во внешней среде.
$m = n + i\kappa$	Комплексный показатель преломления частицы относительно внешней среды, задается <code>--ri Re Im</code> .
d	Направление распространения текущего луча или пучка.
n	Внешняя нормаль грани. В коде знак выбирается так, чтобы ветвление было устойчивым для текущей стороны грани.
θ, ϕ	Углы направления наблюдения в дальней зоне.
$s(\theta, \phi)$	Единичный вектор направления наблюдения.
J	Комплексная матрица Jones 2×2 .
M	Вещественная матрица Mueller 4×4 .
<i>A</i>	Площадь или полигон апертуры; смысл уточняется в конкретной формуле.
N_b	Число выходных пучков для одной ориентации.

Символ	Смысл
N_{ori}	Число ориентаций в усреднении.

2. Координатные системы

2.1. Лабораторная система

Лабораторная система фиксирована относительно падающего света. Обычно падающий луч направлен вдоль одной из осей, а частица поворачивается матрицей Эйлера. Для любого вектора $\mathbf{x}$ в лабораторной системе используются скалярное произведение $\mathbf{a} \cdot \mathbf{b}$ и векторное произведение $\mathbf{a} \times \mathbf{b}$. Все направления лучей нормируются:

$$\|\mathbf{d}\| = 1, \quad \|\mathbf{s}\| = 1, \quad \|\mathbf{n}\| = 1.$$

2.2. Система частицы и повороты Эйлера

Пусть $\mathbf{x}_p$ — координаты вершины в системе частицы, а $\mathbf{x}_l$ — координаты в лабораторной системе. Поворот задается матрицей

$$\mathbf{x}_l = \mathbf{R}(\alpha, \beta, \gamma) \mathbf{x}_p.$$

В коде углы ориентации задаются через наборы beta/gamma или через специальные генераторы ориентаций. Если используется полная SO(3)-параметризация, то добавляется третий угол или кватернион. Для регулярных сеток с осевой симметрией часто достаточно двух углов, потому что поворот вокруг оси падающего луча учитывается аналитически или симметрией частицы.

Удобная запись через элементарные повороты:

$$\mathbf{R}(\alpha, \beta, \gamma) = \mathbf{R}_z(\alpha) \mathbf{R}_y(\beta) \mathbf{R}_z(\gamma).$$

Здесь

$$\mathbf{R}_z(\chi) = \begin{pmatrix} \cos \chi & -\sin \chi & 0 \\ \sin \chi & \cos \chi & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{R}_y(\chi) = \begin{pmatrix} \cos \chi & 0 & \sin \chi \\ 0 & 1 & 0 \\ -\sin \chi & 0 & \cos \chi \end{pmatrix}.$$

Таблица 3: Смысл углов Эйлера.

Символ	Смысл
α	Поворот вокруг лабораторной оси падающего луча или начальный азимут, если используется полная SO(3)-сетка.
β	Наклон оси частицы относительно падающего луча. Именно β имеет синусный якобиан $\sin \beta$.
γ	Поворот частицы вокруг собственной оси после наклона. На β -полюсах все γ геометрически эквивалентны.

2.3. Система грани и апертуры

Для каждого выходного пучка строится локальная система апертуры. Нормаль апертуры $\mathbf{n}_b$ берется из последней грани или из нормали пучка, а две оси в плоскости обозначим $\mathbf{h}_b$ и $\mathbf{v}_b$:

$$\mathbf{h}_b \cdot \mathbf{n}_b = 0, \quad \mathbf{v}_b \cdot \mathbf{n}_b = 0, \quad \mathbf{h}_b \cdot \mathbf{v}_b = 0.$$

Точка $\mathbf{r}$ в лабораторной системе проектируется в координаты апертуры:

$$x = (\mathbf{r} - \mathbf{r}_c) \cdot \mathbf{h}_b, \quad y = (\mathbf{r} - \mathbf{r}_c) \cdot \mathbf{v}_b,$$

где $\mathbf{r}_c$ — центр полигона пучка. Именно эти x, y попадают в BeamEdgeData: массивы вершин, наклоны ребер и признаки валидности ребер.

2.4. Система рассеяния

Для направления наблюдения $\mathbf{s}$ строятся две поперечные оси. В коде используется опорный вектор $\mathbf{v}_f$ и второй вектор

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\|\mathbf{v}_f \times \mathbf{s}\|}.$$

Если направление близко к полюсу и знаменатель мал, код использует специальные ветки для endpoint/pole, потому что базис рассеяния плохо определен при вырождении азимута.

3. Трассировка лучей

3.1. Пересечение луча с гранью

Луч задается начальной точкой $\mathbf{r}_0$ и направлением $\mathbf{d}$:

$$\mathbf{r}(t) = \mathbf{r}_0 + t\mathbf{d}, \quad t \geq 0.$$

Грань лежит в плоскости

$$(\mathbf{r} - \mathbf{p}_f) \cdot \mathbf{n}_f = 0,$$

где $\mathbf{p}_f$ — любая точка грани, $\mathbf{n}_f$ — нормаль грани. Формула пересечения:

$$t_f = \frac{(\mathbf{p}_f - \mathbf{r}_0) \cdot \mathbf{n}_f}{\mathbf{d} \cdot \mathbf{n}_f}.$$

Кандидат допустим, если $t_f > 0$, знаменатель не близок к нулю, а точка $\mathbf{r}(t_f)$ попадает внутрь полигона грани. При невыпуклых частицах этого недостаточно: найденный hit может быть частично или полностью закрыт другой гранью.

3.2. Пучок как полигон

MBS-fast трассирует не одиночный луч, а полигональный пучок. Входной пучок пересекается с гранью, затем его апертура обрезается полигоном грани. Если P_{in} — полигон входного пучка, а F — полигон грани в той же плоскости, то subject-полигон после попадания:

$$P_{\text{hit}} = P_{\text{in}} \cap F.$$

Площадь $|P_{\text{hit}}|$ используется в beam cutoffs и в нормировках. Если полигон вырожден, ветвь удаляется.

3.3. Отражение и преломление

Пусть $\cos A = \mathbf{n} \cdot \mathbf{d}$. В `Splitting.cpp` используется вектор

$$\mathbf{r} = \frac{\mathbf{d}}{\cos A} - \mathbf{n},$$

после чего направление отраженной ветви записано как

$$\mathbf{d}_{\text{refl}} = \frac{\mathbf{r} - \mathbf{n}}{\|\mathbf{r} - \mathbf{n}\|}.$$

Такая запись эквивалентна обычной формуле зеркального отражения после учета знаковой конвенции нормали в коде. Для преломленной ветви код строит

$$s = \frac{1}{m_{\text{eff}}^2 \cos^2 A} - \|\mathbf{r}\|^2, \quad \mathbf{d}_{\text{refr}} = \frac{\mathbf{r}/\sqrt{s} + \mathbf{n}}{\|\mathbf{r}/\sqrt{s} + \mathbf{n}\|}.$$

Если s становится неположительным, обычная преломленная ветвь отсутствует, и включается ветка полного внутреннего отражения.

3.4. Оптический путь

Фаза пучка включает оптический путь

$$\Phi_{\text{path}} = k L_{\text{opt}}.$$

Для внешней среды вклад длины равен геометрической длине. Для внутреннего участка длина умножается на эффективную вещественную часть показателя:

$$L_{\text{opt},\text{in}} \approx \sqrt{\text{Re}(m^2)} L_{\text{geom}}.$$

В поглощающем варианте дополнительно используется мнимая часть показателя и интеграл с затуханием вдоль внутреннего пути.

3.5. Ограничение глубины

Флаг `-n N` ограничивает максимальную глубину внутренних отражений и преломлений. Если g — номер поколения beam tree, то ветви с $g > N$ не трассируются. Это численный контроль стоимости:

$$N_{\text{beam}} \sim 2^g$$

для идеализированной ситуации без cutoffs, хотя на практике дерево сильно режется геометрией и порогами.

4. Коэффициенты Френеля

4.1. Вертикальная и горизонтальная поляризация

Каждое взаимодействие с гранью домножает Jones-матрицу на два комплексных коэффициента: один для локальной вертикальной поляризации, второй для горизонтальной. В коде они называются `cv/ch` или `Tv/Th` в зависимости от ветки. Это не независимые матрицы 4×4 , а диагональные множители в локальном Jones-базисе:

$$\mathbf{J}_{\text{new}} = \begin{pmatrix} c_v & 0 \\ 0 & c_h \end{pmatrix} \mathbf{J}_{\text{old}} \quad \text{или} \quad \mathbf{J}_{\text{old}} \begin{pmatrix} c_v & 0 \\ 0 & c_h \end{pmatrix},$$

в зависимости от внутренней конвенции хранения строк и столбцов.

4.2. Обычное отражение и прохождение

В `ComputeRegularJonesParams` после вычисления $\cos G = \mathbf{n} \cdot \mathbf{d}_{\text{out}}$ используются величины

$$a_0 = m \cos A, \quad a_1 = m \cos G,$$

$$D_v = a_1 + \cos A, \quad D_h = a_0 + \cos G.$$

Для проходящей ветви:

$$c_v^{(T)} = \frac{2a_0}{D_v}, \quad c_h^{(T)} = \frac{2a_0}{D_h}.$$

Для отраженной внутренней ветви:

$$c_v^{(R)} = \frac{\cos A - a_1}{D_v}, \quad c_h^{(R)} = \frac{a_0 - \cos G}{D_h}.$$

Таблица 4: Обозначения в формулах Fresnel regular branch.

Символ	Смысл
m	Комплексный показатель преломления частицы.
$\cos A$	Косинус угла между нормалью и падающим направлением в конвенции кода.
$\cos G$	Косинус угла между нормалью и выходящим направлением.
D_v, D_h	Знаменатели Fresnel для двух локальных поляризаций.
$c_v^{(T)}, c_h^{(T)}$	Множители Jones для прошедшей ветви.
$c_v^{(R)}, c_h^{(R)}$	Множители Jones для отраженной ветви.

4.3. Нормальное падение

При нормальном падении код использует упрощенные коэффициенты:

$$c_T = \frac{2m}{1+m}, \quad c_R = \frac{1-m}{1+m}.$$

Для одной из поляризаций отраженная ветвь получает дополнительный знак, что связано с ориентацией локального поляризационного базиса после отражения:

$$\mathbf{J}_{\text{refl}} \leftarrow \begin{pmatrix} c_R & 0 \\ 0 & -c_R \end{pmatrix} \mathbf{J}.$$

4.4. Полное внутреннее отражение

В ветке complete reflection код строит

$$b = m_{\text{eff}}^2(1 - \cos^2 A) - 1, \quad q = i\sqrt{\max(b, 0)}.$$

Затем

$$c_v = \frac{\cos A - mq}{mq + \cos A}, \quad c_h = \frac{m \cos A - q}{m \cos A + q}.$$

Эти коэффициенты имеют модуль, близкий к единице при непоглощающей среде, но несут фазовый сдвиг. Именно этот фазовый сдвиг важен для поляризационных элементов Mueller.

4.5. Старый знак Fresnel

Флаг `--legacy_sign` включает старую конвенцию знака для forward direction. Этот режим нужен только для воспроизводимости старых результатов. Для новых сравнений CPU/GPU и для физически согласованной ветки следует использовать поведение по умолчанию.

5. Jones-матрицы и диады

5.1. Матрица пучка

Каждый пучок несет

$$\mathbf{J}_{\text{beam}} = \begin{pmatrix} J_{vv} & J_{vh} \\ J_{hv} & J_{hh} \end{pmatrix}.$$

Эта матрица является амплитудной. Поэтому интенсивность связана с квадратами модулей, а интерференция между пучками сохраняется до тех пор, пока Jones-матрицы суммируются до преобразования в Mueller.

5.2. Фазовый множитель

Для внутреннего пучка функция `ComputeFnJones` использует фазу

$$\mathbf{F}_n(\mathbf{s}) = \mathbf{J}_{\text{beam}} \exp \left[i k \left(L_{\text{proj}} - \mathbf{s} \cdot \mathbf{r}_c \right) \right].$$

Здесь L_{proj} хранится как `info.projLenght`, а $\mathbf{r}_c$ — центр пучка. Для external beam берется уже накопленный optical path:

$$\mathbf{F}_n = \mathbf{J}_{\text{beam}} \exp(i k L_{\text{opt}}).$$

5.3. Поворот Jones в базис рассеяния

В `HandlerP0::RotateJones` строятся векторы:

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\|\mathbf{v}_f \times \mathbf{s}\|},$$

$$\mathbf{N}_T = \mathbf{n} \times \mathbf{e}_T, \quad \mathbf{N}_P = \mathbf{n} \times \mathbf{e}_P,$$

$$\mathbf{D}_T = \mathbf{d}_b \times \mathbf{e}_T, \quad \mathbf{D}_P = \mathbf{d}_b \times \mathbf{e}_P.$$

Здесь $\mathbf{e}_T$ — `info.beamBasis`, $\mathbf{e}_P$ — `beam.polarizationBasis`. Далее используется ВАС-САВ identity:

$$\begin{aligned} \mathbf{c}_T &= \mathbf{s} \times \mathbf{N}_T + \mathbf{n} \times \mathbf{D}_T - \mathbf{s} [\mathbf{s} \cdot (\mathbf{n} \times \mathbf{D}_T)], \\ \mathbf{c}_P &= \mathbf{s} \times \mathbf{N}_P + \mathbf{n} \times \mathbf{D}_P - \mathbf{s} [\mathbf{s} \cdot (\mathbf{n} \times \mathbf{D}_P)]. \end{aligned}$$

Матрица поворота:

$$\mathbf{R}_{\text{out}} = \frac{1}{2} \begin{pmatrix} \mathbf{c}_T \cdot \mathbf{v}_t & \mathbf{c}_P \cdot \mathbf{v}_t \\ \mathbf{c}_T \cdot \mathbf{v}_f & \mathbf{c}_P \cdot \mathbf{v}_f \end{pmatrix}.$$

5.4. Полный Jones-вклад пучка

В `ApplyDiffractionFast` итоговый вклад одного пучка:

$$\mathbf{J}_b(\theta, \phi) = F_{\text{edge},b}(\theta, \phi) \mathbf{R}_{\text{out},b}(\theta, \phi) \mathbf{F}_{n,b}(\theta, \phi).$$

Таблица 5: Смысл множителей в полном Jones-вкладе.

Множитель	Смысл
$F_{\text{edge},b}$	Скалярный РО-интеграл по полигональной апертуре пучка b .
$\mathbf{R}_{\text{out},b}$	Проекция/поворот из локального Jones-базиса пучка в базис рассеяния.
$\mathbf{F}_{n,b}$	Накопленная Jones-матрица пучка с фазой optical path и поправкой на направление наблюдения.

5.5. Karczewski-ветка

Флаг `--karczewski` заменяет `RotateJones` на `KarczewskiJones`. В коде эта ветка помечена как экспериментальная. Важное свойство: M_{11} остается тем же, потому что сохраняется Frobenius-норма Jones-оператора:

$$M_{11} = \frac{1}{2} \|\mathbf{J}\|_F^2 = \frac{1}{2} \sum_{i=1}^2 \sum_{j=1}^2 |J_{ij}|^2.$$

Поляризационные элементы M_{33} , M_{34} , M_{44} могут отличаться, потому что меняется структура Jones-матрицы в базисе рассеяния.

6. Преобразование Jones в Mueller

6.1. Полная формула из кода

Пусть

$$\mathbf{J} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

В `src/math/Mueller.cpp` матрица Mueller строится из билинейных комбинаций. Формулы в принятой кодовой конвенции:

$$\begin{aligned} M_{00} &= \frac{|a|^2 + |b|^2 + |c|^2 + |d|^2}{2}, & M_{01} &= \frac{|a|^2 - |b|^2 + |c|^2 - |d|^2}{2}, \\ M_{10} &= \frac{|a|^2 + |b|^2 - |c|^2 - |d|^2}{2}, & M_{11} &= \frac{|a|^2 - |b|^2 - |c|^2 + |d|^2}{2}. \end{aligned}$$

Далее:

$$\begin{aligned} M_{02} &= -\operatorname{Re}(a\bar{b}) - \operatorname{Re}(d\bar{c}), & M_{03} &= \operatorname{Im}(d\bar{c}) - \operatorname{Im}(a\bar{b}), \\ M_{12} &= \operatorname{Re}(d\bar{c}) - \operatorname{Re}(a\bar{b}), & M_{13} &= -\operatorname{Im}(a\bar{b}) - \operatorname{Im}(d\bar{c}), \\ M_{20} &= -\operatorname{Re}(a\bar{c}) - \operatorname{Re}(d\bar{b}), & M_{21} &= \operatorname{Re}(d\bar{b}) - \operatorname{Re}(a\bar{c}), \\ M_{30} &= \operatorname{Im}(a\bar{c}) - \operatorname{Im}(d\bar{b}), & M_{31} &= \operatorname{Im}(d\bar{b}) + \operatorname{Im}(a\bar{c}), \\ M_{22} &= \operatorname{Re}(a\bar{d}) + \operatorname{Re}(b\bar{c}), & M_{23} &= \operatorname{Im}(a\bar{d}) - \operatorname{Im}(b\bar{c}), \\ M_{32} &= -\operatorname{Im}(a\bar{d}) - \operatorname{Im}(b\bar{c}), & M_{33} &= \operatorname{Re}(a\bar{d}) - \operatorname{Re}(b\bar{c}). \end{aligned}$$

Таблица 6: Обозначения в Jones→Mueller.

Символ	Смысл
a, b, c, d	Комплексные элементы итоговой Jones-матрицы для одной detector cell.
$\bar{z}$	Комплексное сопряжение.
$ z ^2 = z\bar{z}$	Квадрат модуля комплексной амплитуды.
$\operatorname{Re}(z), \operatorname{Im}(z)$	Вещественная и мнимая части.
M_{ij}	Элементы Mueller в нулевой индексации кода; в файлах обычно печатаются как $M11, M12, \dots, M44$.

6.2. Когерентная и некогерентная сумма

По умолчанию для одной ориентации:

$$\mathbf{J}_{\text{tot}}(\theta, \phi) = \sum_{b=1}^{N_b} \mathbf{J}_b(\theta, \phi), \quad \mathbf{M}(\theta, \phi) = \mathcal{M}(\mathbf{J}_{\text{tot}}).$$

С флагом --incoh:

$$\mathbf{M}_{\text{incoh}}(\theta, \phi) = \sum_{b=1}^{N_b} \mathcal{M}(\mathbf{J}_b(\theta, \phi)).$$

Разница принципиальная: в первом случае сохраняются интерференционные члены между разными пучками, во втором они исчезают.

6.3. Повороты Mueller

Поворот Stokes-базиса на угол χ действует только на компоненты Q, U :

$$\mathbf{R}_M(\chi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \chi & -\sin \chi & 0 \\ 0 & \sin \chi & \cos \chi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

RightRotateMueller умножает матрицу справа, то есть меняет входной Stokes-базис:

$$\mathbf{M}' = \mathbf{M} \mathbf{R}_M(\chi_{\text{in}}).$$

LeftRotateMueller умножает слева, то есть меняет выходной базис:

$$\mathbf{M}' = \mathbf{R}_M(\chi_{\text{out}}) \mathbf{M}.$$

RotateMueller делает обе операции в одной функции:

$$\mathbf{M}' = \mathbf{R}_M(\chi_{\text{out}}) \mathbf{M} \mathbf{R}_M(\chi_{\text{in}}).$$

7. Дифракция Physical Optics

7.1. Интеграл по апертуре

Для выходного пучка с апертурой A_b скалярная часть РО-дальнего поля записывается как

$$F_b(\mathbf{q}) = \int_{A_b} \exp(i\mathbf{k} \cdot \mathbf{r}) dA, \quad \mathbf{q} = \mathbf{d}_b - \mathbf{s}.$$

7.2. Переход к ребрам полигона

В системе апертуры:

$$A_q = \mathbf{q} \cdot \mathbf{h}_b, \quad B_q = \mathbf{q} \cdot \mathbf{v}_b.$$

Для вершины j :

$$\Phi_j = k(A_q x_j + B_q y_j).$$

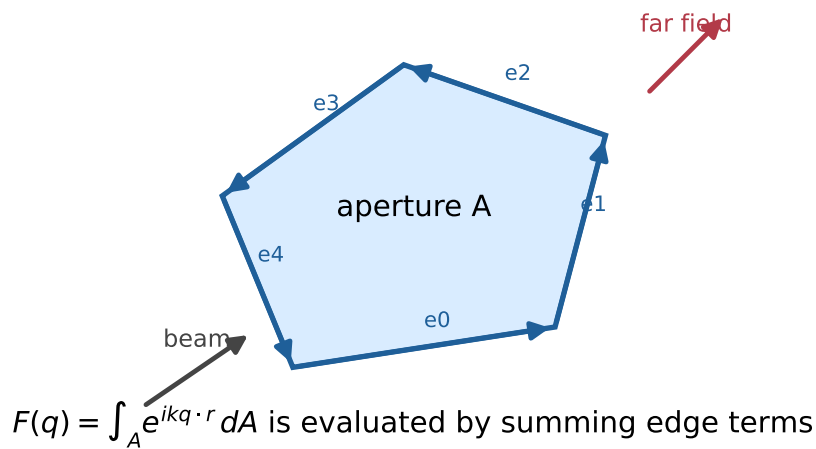


Рис. 2: Aperture integral переводится в сумму по ребрам полигона. Это быстрее и точнее, чем sampling площади.

Если ребро описано через $y = m_j x + c_j$, то один из вариантов edge-суммы имеет вид

$$S_x = \frac{1}{B_q} \sum_j \frac{\exp(i\Phi_{j+1}) - \exp(i\Phi_j)}{A_q + m_j B_q}.$$

Если удобнее параметризовать ребро как $x = \mu_j y + \eta_j$, используется симметричная форма

$$S_y = -\frac{1}{A_q} \sum_j \frac{\exp(i\Phi_{j+1}) - \exp(i\Phi_j)}{A_q \mu_j + B_q}.$$

В коде выбор между ветками зависит от того, какой знаменатель устойчивее. Для малых A_q и B_q используется предельная формула площади:

$$F_b(0) \approx C_k |A_b|,$$

где C_k — комплексный нормировочный множитель, соответствующий выбранной форме интеграла в коде.

Таблица 7: Символы edge-интеграла.

Символ	Смысл
q	Разность направления пучка и направления наблюдения.
A_q, B_q	Компоненты q в плоскости апертуры.
(x_j, y_j)	Вершина полигона пучка в системе апертуры.
m_j, μ_j	Наклоны ребра: dy/dx и dx/dy .

Символ	Смысл
Φ_j	Фаза комплексной экспоненты в вершине j .
S_x, S_y	Две эквивалентные формы edge-суммы, выбираемые по устойчивости.

7.3. Почему важны точки 0 и 180 градусов

В forward/backward направлениях часть базисов и знаменателей становится вырожденной или почти вырожденной:

$$\|\mathbf{v}_f \times \mathbf{s}\| \rightarrow 0, \quad A_q \rightarrow 0, \quad B_q \rightarrow 0.$$

Поэтому в этих точках особенно важны:

- одинаковая обработка endpoint в CPU и GPU;
- одинаковая формула предела edge-интеграла;
- одинаковый вес γ на beta-полюсах при --pole;
- отсутствие расхождения в знаке Fresnel и фазовой конвенции;
- одинаковая точность: float может заметно отличаться от double именно в near-singular ветках.

7.4. Поглощение

Если $\text{Im}(m) > 0$ или указан --abs, внутренняя часть пучка получает затухание:

$$\exp(ikmL) = \exp(ik \text{Re}(m)L) \exp(-k \text{Im}(m)L).$$

Тогда вклад длинных внутренних путей уменьшается. В численном смысле это помогает сходимости дерева лучей, но меняет физическую интенсивность и поляризационные элементы.

8. Clipping невыпуклых частиц и агрегатов

8.1. Зачем нужен clipping

Для выпуклой частицы пересечение пучка с ближайшей гранью обычно однозначно. Для невыпуклой частицы другая грань может закрывать часть апертуры.

Тогда надо вычесть проекцию блокирующей грани из subject-полигона:

$$P_{\text{vis}} = P_{\text{subj}} \setminus \bigcup_{m=1}^{N_c} \Pi_{\text{subj}}(C_m).$$

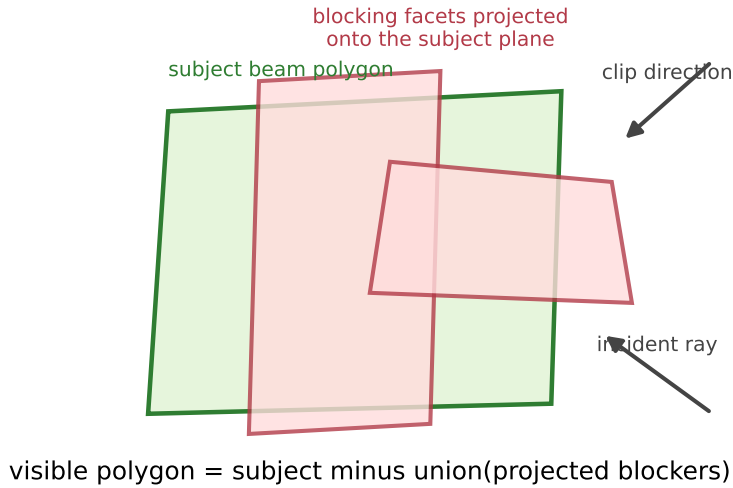


Рис. 3: Невыпуклый clipping: projected blocker удаляет часть subject-полигона.

Таблица 8: Объекты clipping.

СИМВОЛ	СМЫСЛ
P_{subj}	Полигон текущего пучка, который проверяется на видимость.
C_m	Кандидат-блокер: грань или полигон, потенциально закрывающий subject.
Π_{subj}	Проекция blocker на плоскость subject вдоль направления распространения.
P_{vis}	Оставшийся видимый полигон после polygon difference.
N_c	Число кандидатов после быстрых фильтров.

8.2. Агрегаты

В агрегате blocking candidate может принадлежать другой компоненте:

$$C_m \in \bigcup_{p=1}^{N_{\text{part}}} \mathcal{F}_p,$$

где $\mathcal{F}_p$ — множество граней компоненты p .

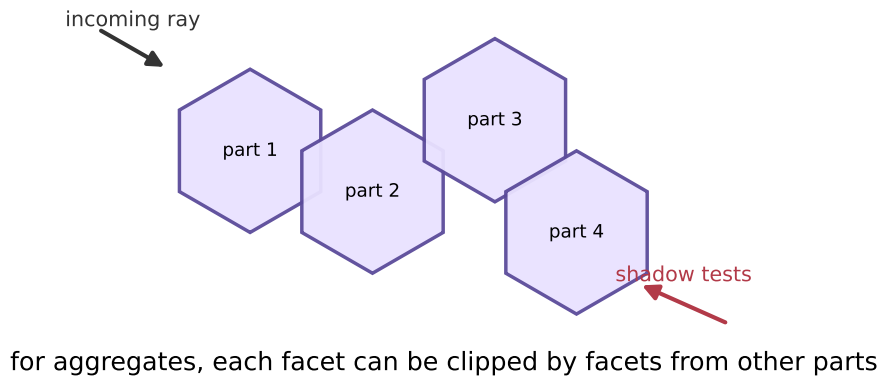


Рис. 4: Для агрегата затенение идет не только внутри одной компоненты, но и между разными компонентами.

8.3. Projected-AABB prefilter

Точный polygon difference дорогой. Поэтому `--trace_prefilter` сначала строит bounding box проекции blocker:

$$B(C_m) = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}].$$

Если

$$B(C_m) \cap B(P_{\text{subj}}) = \emptyset,$$

то точный clipping не нужен. Это не меняет физику, а только уменьшает число дорогих операций.

9. Размер частицы и масштабирование

9.1. Built-in и file particles

Частица задается либо `-p TYPE PARAMETERS...`, либо `--pf FILE`. Встроенные частицы создаются в коде, file particle читается из файла с геометрией. Нельзя одновременно задавать оба источника геометрии. Флаг `--save-geometry FILE` (alias `--save-particle`) сохраняет фактически используемую геометрию после масштабирования и выбора `convex/nonconvex` режима, после чего расчет продолжается.

Таблица 9: Основные встроенные частицы.

Type	Форма	Параметры
1	Hexagonal column/plate	L D : осевая длина и диаметр между противоположными вершинами. Отношение L/D определяет column или plate.
2	Bullet	L D : полная осевая длина и диаметр; высота пирамидальной головки вычисляется из фиксированного угла 62° .
3	Bullet rosette	L D [CAP]: шесть bullets; без CAP используется та же конструкция 62° .
4	Droxtal	SCALE: масштаб фиксированного шаблона; legacy-форма L D SCALE принимает, но игнорирует L, D .
10	Concave hexagonal	L D CAVITY_DEG: верхняя и нижняя пирамидальные полости; угол должен быть положительным и меньше $\arctan(L/D)$.
12	Двухкомпонентный hexagonal aggregate	L D 2: ровно две одинаковые восьмигранные колонки в фиксированном взаимном положении.
999	Fixed built-in aggregate	SCALE: масштаб фиксированной регрессионной aggregate-геометрии.

9.2. Native particle format и экспорт геометрии

Первые три непустые строки имеют строгий смысл:

```

CONCAVE
AGGREGATE [FACETS_PER_PART]
BETA_DOMAIN_DEG GAMMA_DOMAIN_DEG

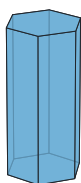
x0 y0 z0
x1 y1 z1
x2 y2 z2

x0 y0 z0 # следующая грань
...

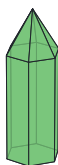
```

CONCAVE равен 0 или 1. AGGREGATE равен 0 для одной частицы, либо записывается

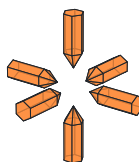
1 Hexagonal column
L=2, D=1
convex



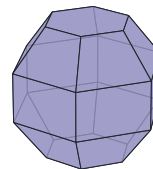
2 Bullet
L=2, D=1; cap=auto
convex



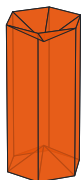
3 Bullet rosette
L=2, D=1; cap=auto
nonconvex, aggregate



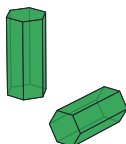
4 Droxtal
scale=1
convex



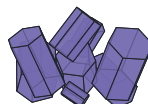
10 Concave column
L=2, D=1; cavity=30 deg
nonconvex



12 Two-column aggregate
L=2, D=1; parts=2
nonconvex, aggregate



999 Fixed aggregate
scale=1
nonconvex, aggregate



Rendered from the native files
written by --save-geometry.

D: vertex-to-vertex hexagon diameter
L: prism or branch length
scale: uniform geometry scale

Рис. 5: Все текущие built-in формы. Изображения построены из файлов examples/particles, записанных самими constructors программы.

как 1 FACETS_PER_PART для компонентов с одинаковым числом граней. Третья строка задает ширину фундаментальной области по beta и gamma в градусах. Пустая строка разделяет грани; у каждой грани должно быть не меньше трех конечных неколлинеарных вершин. Символ ## начинается полный или inline comment. Порядок вершин задает нормаль грани, поэтому winding всех граней должен быть согласован.

Экспорт пишет координаты с 17 значащими цифрами и проверяет aggregate metadata. Файл содержит уже масштабированную effective geometry, поэтому подходит для точного повторения расчета, визуализации clipping и round-trip проверки:

```
gpu/bin/mbs_po_gpu_double_fast --po -p 10 20 12 15 \
  --save-geometry concave.particle ...
gpu/bin/mbs_po_gpu_double_fast --po --pf concave.particle ...
```

Автоматический particle_for_check.dat сохраняется для совместимости; --save-geometry удобнее, когда нужен заранее известный стабильный путь.

9.3. Equivalent radius и k_{eq}

Эквивалентный радиус по объему:

$$r_{eq} = \left(\frac{3V}{4\pi} \right)^{1/3}.$$

Эквивалентный размерный параметр:

$$k_{eq} = \frac{2\pi r_{eq}}{\lambda}.$$

Если файл частицы масштабируется по --k_eq X, то масштаб s выбирается из

$$s = \frac{k_{eq,target}\lambda}{2\pi r_{eq,old}}.$$

Таблица 10: Флаги размера.

Флаг	Смысл
--rs SIZE	Масштабировать file particle так, чтобы $D_{max} = SIZE$.
--k_eq X	Масштабировать так, чтобы $k_{eq} = X$. Требуется -w.
--multigrid A B N	Логарифмический диапазон D_{max} ; canonical name --dmax-grid.

Флаг	Смысл
--multikeq A B	Логарифмический диапазон k_{eq} ; canonical name
N	--k-eq-grid.
--multikeq_list	Точный список положительных k_{eq} из файла.
FILE	

10. Ориентационное усреднение

10.1. Интеграл по ориентациям

Для случайных ориентаций финальная матрица:

$$\overline{\mathbf{M}}(\theta) = \frac{1}{W} \sum_{i=1}^{N_{\text{ori}}} w_i \mathbf{M}_i(\theta), \quad W = \sum_{i=1}^{N_{\text{ori}}} w_i.$$

В непрерывной форме для двух углов:

$$\overline{\mathbf{M}}(\theta) = \frac{1}{4\pi} \int_0^{2\pi} \int_0^\pi \mathbf{M}(\theta; \beta, \gamma) \sin \beta \, d\beta \, d\gamma.$$

Удобная переменная $\mu = \cos \beta$ дает

$$\sin \beta \, d\beta = -d\mu,$$

поэтому равномерная сетка по μ физически естественнее, чем равномерная сетка по β .

10.2. Регулярная beta/gamma сетка

Если beta cell имеет границы $\beta_{j-1/2}$ и $\beta_{j+1/2}$, ее вес:

$$w_\beta(j) = \cos \beta_{j-1/2} - \cos \beta_{j+1/2}.$$

Если gamma равномерна:

$$w_\gamma = \frac{\gamma_{\text{max}} - \gamma_{\text{min}}}{N_\gamma}.$$

Итоговый вес:

$$w_{j,l} = w_\beta(j) w_\gamma(l).$$

10.3. Флаг --pole

На beta-полюсах $\beta = 0$ и $\beta = \pi$ все γ описывают одну и ту же физическую ориентацию:

$$\mathbf{R}_z(\gamma)\mathbf{R}_y(0) = \mathbf{R}_z(\gamma),$$

а вращение вокруг направления падающего света может быть эквивалентно азимутальному повороту результата. Поэтому --pole оставляет одну gamma-точку на полюсе, но вес должен соответствовать целому gamma-интервалу:

$$w_{\text{pole}} = w_{\beta}(\gamma_{\text{max}} - \gamma_{\text{min}}).$$

Именно из-за этой специальной обработки CPU/GPU различия раньше проявлялись в точках 0° и 180° : endpoint, pole-weight и near-singular scattering basis должны совпадать побитово по логике ветвления, иначе M_{11} и слабые поляризационные элементы расходятся.

10.4. Oldauto

--oldauto DIV строит регулярную сетку из оценки дифракционного углового масштаба:

$$\Delta\theta_{\text{diff}} = 0.69 \frac{\lambda}{D_{\text{max}}} \frac{180}{\pi}.$$

Параметр DIV делит этот шаг:

$$\Delta\theta_{\text{ori}} = \frac{\Delta\theta_{\text{diff}}}{\text{DIV}}.$$

Дальше число beta/gamma выбирается так, чтобы покрыть область симметрии с этим шагом. Практические значения: 2, 4, 8. Чем больше DIV, тем больше ориентаций и тем дороже расчет.

10.5. Sobol, Hammersley, lattice, Monte Carlo

Квазислучайные режимы заменяют регулярную сетку набором точек низкой дискрепантности. Идея:

$$\overline{\mathbf{M}}_N = \frac{1}{N} \sum_{i=1}^N \mathbf{M}(T(\mathbf{u}_i)),$$

где $\mathbf{u}_i \in [0, 1)^d$ — Sobol/Hammersley/lattice point, а T переводит единичный куб в углы ориентации с правильной мерой.

Таблица 11: Ориентационные методы.

Флаг	Метод
--fixed B G	Одна ориентация beta/gamma, удобно для отладки CPU/GPU.
--orientfile FILE	Ориентации из файла.
--random NB NG	Явная регулярная beta/gamma сетка; canonical name
	--euler-grid.
--oldauto DIV	Production-сетка из дифракционного масштаба.
--sobol N	Sobol quasi-random orientations.
--sobol_seed N	Sobol с Owen scrambling seed.
SEED	
--so3_quat N	Hammersley/SO(3) через кватернионы.
--hammersley N	Hammersley orientation set.
--lattice N	Rank-1 lattice orientation set.
--montecarlo N	Псевдослучайные ориентации; ошибка обычно падает как $N^{-1/2}$.

10.6. Adaptive и формула выбора числа ориентаций

При сравнении кандидатов по N_ϕ , глубине отражений и Euler-сетке для двух приближений A и B ошибка одной theta-строки равна

$$e_{\text{row}} = \max \left(\frac{|M_{11}^A - M_{11}^B|}{\max(|M_{11}^A|, |M_{11}^B|, \varepsilon_0)}, \max_{(i,j) \neq (1,1)} \frac{|R_{ij}^A - R_{ij}^B|}{\max(1, |R_{ij}^A|, |R_{ij}^B|)} \right), \quad R_{ij} = \frac{M_{ij}}{M_{11}}.$$

Здесь M_{ij} — элемент матрицы Мюллера, R_{ij} — тот же элемент, нормированный на M_{11} , а ε_0 — малое защитное число. Затем берется максимум по theta-строкам и добавляется относительное изменение интеграла $\int M_{11} d\Omega$. Для глубины внутренних отражений дополнительно проверяется выходная энергия. Theta-поиск использует другой тест: в точках 1/3 и 2/3 каждого интервала реальная матрица сравнивается с интерполяцией, после чего отдельно проверяется интеграл M_{11} . Поиск числа Sobol-ориентаций проверяет входную энергию и все 16 элементов на контрольных theta-строках текущей сетки.

По умолчанию для orientation, phi, reflection и Euler-поиска нужны два последовательных успешных уточнения; theta-поиск уточняет интервалы напрямую. --convergence-passes N меняет число только для этих streak-проверок. Достижение верхнего предела является ошибкой для phi, reflection, theta, Euler и unified-auto режимов. Отдельный --adaptive-orientations оставлен exploratory-режимом: на верхнем пределе он выдает предупреждение и записывает лучший результат; внутри unified-auto та же проверка является строгой.

10.7. Файл настроек адаптивной сходимости

Флаг `--adaptive-config FILE` загружает все production-пределы из одного текстового файла. Полный шаблон с пояснением каждой строки находится в `configs/adaptive.example.conf`. Формат строгий: `key = value`; после символа `#` или `;` разрешен комментарий. Например:

```
reflections.min = 2
reflections.max = 30
reflections.tolerance = 0.01 # 1 percent
alpha.min_points = 12
alpha.max_points = 2400
alpha.tolerance = 0.01
orientations.min = 128
orientations.max = 262144
orientations.tolerance = 0.02
```

Отдельные группы `reflections`, `alpha`, `theta`, `orientations`, `beta` и `gamma` задают минимум, максимум и свою относительную точность. Группа `controller` задает число последовательных успешных уточнений, диапазон `pilot`-набора и максимальное число совместных проходов. Значение `0.01` означает 1%, а не 0.01%.

Если индивидуальная точность отсутствует в файле, используется общий EPS из `--auto`, `--autofull` или другого выбранного режима. Индивидуальная точность из файла имеет приоритет над общим EPS. Явные флаги командной строки `--max-*`, `--max-reflections` и `--convergence-passes` имеют приоритет над файлом. Неизвестный или повторный ключ, неверный диапазон и неправильный тип значения останавливают запуск до расчета с указанием файла, номера строки и способа исправления.

```
cpu/bin/mbs_po_mpi --method po --particle 1 100 70 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --autofull 0.02 \
  --adaptive-config configs/adaptive.example.conf \
  --backend cpu --threads 64 --output converged --close
```

10.8. Auto, autofull, oldautofull

Параметры не являются строго независимыми. Глубина n меняет дерево лучей; N_ϕ меняет азимутальное усреднение, по которому обнаруживаются особенности по θ ; число ориентаций меняет каждую усредненную строку. Поэтому общий порядок имеет вид

$$n \longrightarrow N_\phi \longrightarrow \{\theta_j\} \longrightarrow N_{\text{orient}}.$$

--auto EPS оставляет n фиксированным и подбирает N_ϕ , theta grid и Sobol N .
 --autofull EPS дополнительно подбирает n . --oldautofull EPS теперь заканчивает не эвристической oldauto-сеткой, а регулярной квадратурой Эйлера: N_β и N_γ уточняются по очереди, после чего оба увеличиваются совместно. Весь цикл повторяется на общем увеличенном pilot-наборе, пока pilot, следующий из выбранного числа ориентаций, и все параметры не образуют совместную неподвижную точку.

В unified-auto явный --phi-points N фиксирует N_ϕ , а --scattering-grid или --theta-grid-file фиксирует theta-сетку; остальные параметры продолжают подбираться. Чтобы искать эту размерность, соответствующий явный флаг надо убрать. Отдельные --adaptive-phi и --auto-phi по-прежнему несовместимы с --phi-points, потому что их единственная задача — подобрать N_ϕ .

--auto_tgrid EPS проверяет две внутренние точки каждого theta-интервала по полной матрице Мюллера и по интегралу M_{11} . В алгоритме нет фиксированных областей 22° и 46° : особенности конкретной формы должны обнаруживаться из рассчитанной кривой. Практически:

$$\text{стоимость} \approx N_{\text{ori}} N_\theta N_\phi N_b.$$

Поэтому автоматический режим может быть сильно дороже одной ручной production-сетки, но он полезен для выбора параметров.

Каждая попытка записывается в файл <output>/<name>_convergence.tsv: параметр, номер совместного прохода, кандидат, максимальная ошибка, худший theta-угол, номер элемента и статус.

10.9. Почему симметрия столбика не делит Nphi на шесть

В коде используется соглашение

$$\mathbf{R} = R_z(\alpha) R_y(\beta) R_z(\gamma).$$

Цикл по азимуту рассеяния за счет вращательной ковариантности реализует усреднение по лабораторному углу α , поэтому в этом контексте $N_\phi = N_\alpha$. У шестигранного столбика есть осевая симметрия $R_z(2\pi/6)$ в системе частицы. Она уже сокращает область γ до

$$0 \leq \gamma < \frac{\pi}{3} = 60^\circ.$$

Но при $\beta \neq 0, \pi$ наклоненный столбик не инвариантен относительно поворота на 60° вокруг лабораторной оси падающего луча. Поэтому для α остается область $0 \leq \alpha < 2\pi$, и автоматическое деление N_ϕ на шесть было бы физически неверным. --alpha-points является понятным алиасом --phi-points, а --adaptive-alpha — алиасом --adaptive-phi; они не включают дополнительное приближение.

11. Scattering grid

11.1. Theta rows и solid angle

Uniform theta grid с --grid T0 T1 NPHI NTH печатает $N_\theta + 1$ строк. Вес строки:

$$\Delta\Omega_j = 2\pi [\cos \theta_{j,\text{left}} - \cos \theta_{j,\text{right}}] .$$

Для полного диапазона $0^\circ \dots 180^\circ$:

$$\sum_j \Delta\Omega_j = 4\pi .$$

11.2. Azimuthal grid

Для полного output используется N_ϕ азимутальных точек. Для случайно ориентированных частиц часто печатается результат, интегрированный по ϕ , но внутри PO и GPU kernels ϕ остается отдельным измерением работы:

$$N_{\text{work}} \sim N_b N_\theta N_\phi N_{\text{ori}} .$$

11.3. Auto theta grid

Forward peak требует более мелкого шага, чем боковые и задние углы. Автоматическая theta-сетка использует оценку ширины пика:

$$\theta_{\text{peak}} \approx \frac{180^\circ}{x}, \quad x = \frac{2\pi r}{\lambda} .$$

Затем строятся зоны: тонкая near-forward сетка, переходная сетка, средняя область и более грубая backscattering область. Это уменьшает число строк без потери переднего пика.

12. CPU-реализация и AVX

12.1. Что параллелится на CPU

CPU использует OpenMP/MPI по ориентациям, theta rows, beams и size batches. Основные дорогие операции:

$$\Phi = k(A_q x + B_q y), \quad \exp(i\Phi) = \cos \Phi + i \sin \Phi,$$

edge-сумма по ребрам и накопление Jones/Mueller.

12.2. AVX-512 packing

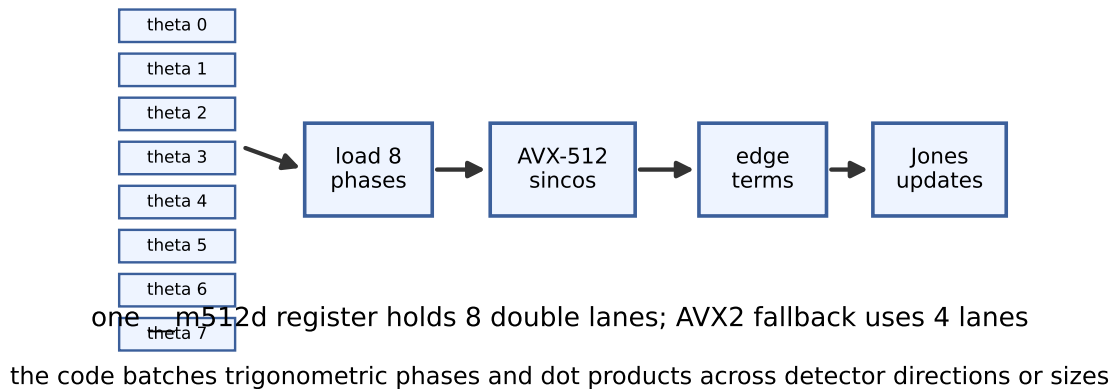


Рис. 6: AVX-512 упаковывает восемь double фаз в один vector register; AVX2 упаковывает четыре.

Для AVX-512 double:

$$\Phi = (\Phi_0, \Phi_1, \dots, \Phi_7),$$

$$C = \cos \Phi, \quad S = \sin \Phi.$$

Одна vector-инструкция или vectorized polynomial path обрабатывает сразу восемь detector directions или size values. Для AVX2:

$$\Phi_{AVX2} = (\Phi_0, \Phi_1, \Phi_2, \Phi_3).$$

12.3. Почему важны флаги компиляции

Если компилятор не имеет права генерировать AVX-512, быстрый путь не включится. Для EYUC:

Таблица 12: Рекомендуемые CPU-флаги.

Платформа	ARCH_FLAGS	Комментарий
EYUC Zen2	-march=znver2 -mtune=znver2	AVX2/FMA.
EYUC Zen3	-march=znver3 -mtune=znver3	AVX2/FMA.

Платформа	ARCH_FLAGS	Комментарий
EPYC Zen4	-march=znver4 -mtune=znver4	AVX-512 при поддержке GCC/Clang.
EPYC Zen5	-march=znver5 -mtune=znver5	Лучший вариант для новых компиляторов.
Portable AVX2	-O3 -mavx2 -mfma	Для переносимого бинарника под AVX2.
Explicit AVX-512	-O3 -mavx512f -mavx512dq -mavx512cd -mavx512bw -mavx512vl	Только если <code>lscpu</code> показывает эти флаги.

Флага AVX12 у GCC/Clang нет. Если имеется в виду EPYC Zen5 с AVX-512, используйте `-march=znver5`; если компилятор старый, используйте явные `-mavx512*` после проверки `lscpu`.

13. GPU-реализация

13.1. Что ускоряет GPU

GPU быстрее, потому что РО-дифракция почти идеально раскладывается на независимые работы:

$$(b, \theta, \phi, o) \mapsto \mathbf{J}_{b,o}(\theta, \phi),$$

где b — beam, o — orientation. До момента когерентной суммы каждый вклад независим.

Таблица 13: GPU kernels и смысл.

Путь	Что делает thread	Когда выгодно
Atomic Jones	Считает вклад beam/grid и делает <code>atomicAdd</code> в complex Jones buffer.	General fallback и diagnostic modes.
No-atomic grid	Thread владеет output cell и сам проходит по beams.	Когда layout позволяет избежать конкуренции writers.
Fused Mueller	Thread считает Jones локально и сразу переводит/добавляет Mueller.	Production full-only output.

Путь	Что делает thread	Когда выгодно
Staged orientation	Сначала per-orientation Mueller, потом reduction.	Большие batches, меньше contention.
Multi-k_eq fused	Несколько близких размеров из общего beam batch.	Size scans, если геометрия и сетка совместимы.

13.2. Atomic accumulation

Jones-матрица имеет четыре комплексных элемента:

$$\mathbf{J} = \begin{pmatrix} J_{00} & J_{01} \\ J_{10} & J_{11} \end{pmatrix}, \quad J_{pq} = x_{pq} + iy_{pq}.$$

Atomic path делает восемь атомарных сложений:

$$\text{atomicAdd}(x_{pq}), \quad \text{atomicAdd}(y_{pq}), \quad p, q \in \{0, 1\}.$$

Это корректно для когерентной суммы, но порядок сложения недетерминирован. Поэтому последние биты могут отличаться от CPU, особенно в float и в точках near-singular.

13.3. Почему fused path быстрее

Fused kernel избегает записи полного Jones buffer в глобальную память:

$$\sum_b \mathbf{J}_b \rightarrow \mathcal{M} \left(\sum_b \mathbf{J}_b \right) \rightarrow \text{global Mueller}.$$

Меньше global memory traffic, меньше atomics, выше arithmetic intensity. Это особенно важно при большом N_b и большом N_ϕ .

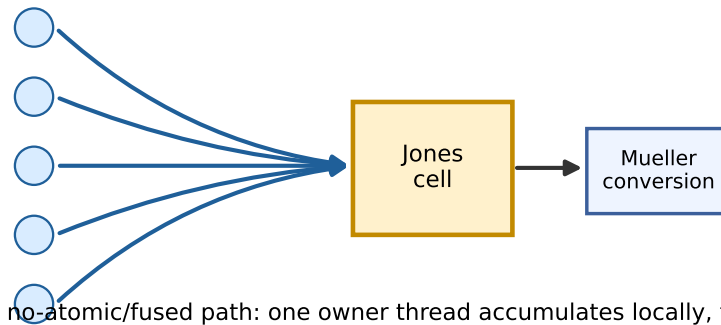
Отдельное ускорение по azimuth дает cuFFT path. Для обычного запуска используется --fft, для явного и воспроизводимого сжатия --fft-factor N, а для измеряемого nested-grid контроля --fft-tolerance EPS. Алгоритм, цена дополнительной проверки и смысл оценки ошибки разобраны в разделе 19.11.

13.4. Multi-GPU

Multi-GPU делит работу по независимым блокам: orientations, theta chunks или size batches. Если устройства $g = 1, \dots, G$ получают веса W_g , идеальный speedup:

$$T_G \approx \frac{T_1}{G} + T_{\text{copy}} + T_{\text{reduce}}.$$

atomic path: many beam threads add into one coherent Jones cell



no-atomic/fused path: one owner thread accumulates locally, then writes once

Рис. 7: CUDA paths: atomic Jones accumulation, no-atomic ownership и fused diffraction-to-Mueller.

На практике ограничивают PCIe/NVLink transfers, разная скорость GPU, размер batches и необходимость финальной редукции Mueller.

Таблица 14: GPU environment variables.

Переменная	Смысл
CUDA_VISIBLE_DEVICES=0,1	Выбрать доступные GPU.
MBS_GPU_FUSED_MUELLER=1	Предпочитать fused diffraction-to-Mueller kernels.
MBS_GPU_STAGE_MUELLER=1	Включить staged per-orientation Mueller reduction.
MBS_GPU_MULTI=0	Отключить automatic multi-orientation GPU batching.
MBS_GPU_MULTI_K_FULL=1	Experimental fused multi-k _{eq} diffraction.
MBS_FFT_PHI_FACTOR=N	Legacy FFT factor, только если не задан --fft-factor; CLI имеет приоритет.
MBS_SHARED_ORIENT_CHUNK=N	Переопределить размер orientation chunk.
MBS_BEAM_LOG=PATH	Явно записать первый обработанный набор РО-пучков; по умолчанию beam log не создается.

14. Сборка

14.1. CPU

```
make -C cpu -j

OMP_NUM_THREADS=64 cpu/bin/mbs_po_mpi --po -p 1 125.9 78.09 \
--ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
--grid 0 180 600 180 --threads 64 --close -o out_cpu
```

14.2. GPU

```
PATH=/usr/local/cuda/bin:$PATH make -C gpu -j
make -C gpu float -j
make -C gpu float_fast -j
make -C gpu double_fast -j
```

Таблица 15: GPU targets.

Target	Binary	Точность	Когда использовать
float	mbs_po_gpu_float	float	Быстро, без fast math.
float_fast	mbs_po_gpu_float_fast	float	Максимальная скорость, возможны отличия от double.
double_fast	mbs_po_gpu_double_fast	double	Сравнения CPU/GPU, 0/180 градусов, production validation.

15. Запуск и диагностика

15.1. Минимальный production-пример

```
gpu/bin/mbs_po_gpu_double_fast --po -p 1 125.9 78.09 \
--ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
--grid 0 180 600 180 --threads 64 --close -o hex_gpu_double
```

15.2. CPU из GPU binary

```
gpu/bin/mbs_po_gpu_double_fast --po --cpu -p 1 125.9 78.09 \
--ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
--grid 0 180 600 180 --threads 64 --close -o hex_cpu_from_gpu_bin
```

15.3. Проверка точек 0 и 180 градусов

```
gpu/bin/mbs_po_gpu_double_fast --po -p 1 125.9 78.09 \
--ri 1.3116 0 -w 0.532 -n 1 --oldauto 2 --pole \
--grid 0 180 600 1 --threads 64 --close -o check_0_180
```

16. Краткий справочник флагов

16.1. Метод и частица

Таблица 16: Метод, геометрия, физические параметры.

Флаг	Аргументы	Описание
--po	нет	Physical Optics: трассировка + дифракция.
--go	нет	Только geometrical optics, без РО-интеграла.
-p	TYPE PARAMETERS...	Built-in particle; точный синтаксис зависит от TYPE.
--pf	FILE	Частица из файла.
--save-geometry	FILE	Сохранить effective native geometry и продолжить; alias --save-particle.
--rs	SIZE	Масштабировать file particle по $D_{\max}$.
--k_eq	X	Масштабировать по equivalent size parameter.
--ri	Re Im	Комплексный показатель преломления.
-w	LAMBDA	Длина волны.
-n	N	Максимальная глубина трассировки.
--abs	нет	Включить absorption integral.

Флаг	Аргументы	Описание
--close	нет	Завершить процесс после расчета; compatibility-флаг.

16.2. Ориентации

Таблица 17: Флаги ориентаций.

Флаг	Аргументы	Описание
--fixed	BETA GAMMA	Одна ориентация в градусах.
--orientfile	FILE	Загрузить ориентации из файла.
--random, --euler-grid	NB NG	Явная beta/gamma grid.
--oldauto	DIV	Physics-based grid из diffraction scale.
--ring_points	N	Точек на diffraction ring для oldauto estimates.
--sobol	N	Sobol orientations.
--sobol_seed	N SEED	Sobol с Owen seed.
--owen_seeds	LIST	Набор Owen seeds.
--so3_quat	N	SO(3) quaternion orientations.
--hammersley	N	Hammersley orientations.
--lattice	N	Rank-1 lattice orientations.
--montecarlo	N	Random orientations.
--adaptive_euler	EPS	Раздельный подбор N_β и N_γ с совместной проверкой.
--adaptive	EPS	Подбор только числа Sobol-ориентаций.
--auto	EPS	Auto phi, theta, Sobol orientations при фиксированном n .
--autofull	EPS	Auto -n, theta, phi, orientations.
--oldautofull	EPS	Auto search, финальная адаптивная регулярная Euler grid.
--adaptive-config	FILE	Загрузить min/max/точность каждого adaptive-параметра.
--maxorient	N	Верхняя граница adaptive orientations.
--max_beta_points	N	Верхняя граница N_β .
--max_gamma_points	N	Верхняя граница N_γ .
--chunk	N	Max orientations/gamma values per memory chunk.

Флаг	Аргументы	Описание
--mirror_gamma	нет	Делит default gamma-domain пополам; явный gamma-range считается уже сокращенным.
--pole	нет	Считать beta-poles как одну gamma с полным весом.
--coh_orient	нет	Legacy coherent-across-orientations mode.

16.3. Сетка, ускорение, cutoffs

Таблица 18: Grid, acceleration, cutoffs.

Флаг	Аргументы	Описание
--grid	T0 T1 NPHI NTH	Uniform scattering grid; выводится $NTH + 1$ строк.
--auto_tgrid	EPS	Adaptive theta grid по полной Mueller-ошибке, без фиксированных halo-углов.
--auto_phi	нет	Измеряемая сходимость N_ϕ с допуском 0.02.
--adaptive_phi	EPS	Подбор только N_ϕ .
--adaptive_reflections	EPS	Подбор только глубины n .
--max_theta_points	N	Верхняя граница числа theta-точек.
--max_phi_points	N	Верхняя граница N_ϕ .
--convergence_passes	N	Streak для orientation/phi/reflection/Euler; не для theta.
--gpu	нет	Использовать CUDA backend.
--cpu	нет	Принудительно CPU backend из GPU binary.
--threads	N	OpenMP threads.
--fft	нет	cuFFT phi interpolation с автоматическим factor.
--fft-factor	N	Явное сжатие direct/output phi, 1 ... 64; также включает FFT.

Флаг	Аргументы	Описание
--fft-tolerance	EPS	Nested-grid оценка при $0 < \text{EPS} < 1$ и одно уточнение doubled direct grid.
--cutoff-profile	MODE	Полный preset: off, safe, balanced или fast.
--beam-cutoff	EPS	Legacy output shortcut: Jones OR area.
--beam-cutoff-jones	EPS	Output cutoff по относительно-му $ J ^2$.
--beam-cutoff-area	EPS	Output cutoff по площади beam polygon.
--beam-cutoff-importance	EPS	Output cutoff по $ J ^2 A$.
--trace-cutoff	EPS	Legacy internal shortcut: Jones OR area.
--trace-cutoff-jones	EPS	Internal pruning по относительно-му $ J ^2$.
--trace-cutoff-area	EPS	Internal pruning по площади.
--trace-cutoff-importance	EPS	Internal pruning по $ J ^2 A$.
--beam-area-ratio	R	Small-fragment ratio $R \geq 1$; legacy default 100.
--trace-max-beams	N	Hard cap дерева на ориентацию; 0 отключает configured cap.
--trace_prefilter	нет	Projected-AABB prefilter for non-convex clipping.
--multigrid	A B N	Log-grid абсолютных физических $D_{\max}$; для каждой точки $x = \pi D_{\max}/\lambda$, независимо от размера исходной формы.
--multikeq, --multikeq_list	range/file	Диапазон или список k_{eq} .

16.4. Output, диагностика, legacy

Таблица 19: Output и диагностические флаги.

Флаг	Аргументы	Описание
-o	PREFIX	Prefix output files.
--save_betas	нет	Save per-beta Mueller files.

Флаг	Аргументы	Описание
--full_only	нет	Писать только full Mueller output; default.
--noshadow_output	нет	Дополнительно писать _noshadow.
--shadow	нет	Legacy flag, сейчас без эффекта.
--incoh	нет	Incoherent per-beam Mueller sum.
--jones	нет	Писать Jones matrices, где поддержано.
--karczewski	нет	Experimental Karczewski polarization matrix.
--legacy_sign	нет	Старая Fresnel sign convention.
--help	нет	Common options.
--help-debug	нет	Полный список debug/experimental options.

17. Выходные файлы

Основной файл содержит theta, вес solid angle и элементы Mueller. В текстовом выводе элементы обычно называются $M_{11}, \dots, M_{44}$, что соответствует математическим $M_{00}, \dots, M_{33}$ в нулевой индексации кода.

Таблица 20: Типовые output objects.

Объект	Смысл
theta	Scattering angle in degrees.
2pi*dcos	Вес solid-angle ring для строки theta.
Mij	Элементы Mueller после beam sum и orientation average.
*_noshadow	Output без shadow/external beam, если включен --noshadow_output.
*_betas/	Per-beta intermediate outputs, если включен --save_betas.
checkpoint	Resume files для длинных orientation-grid runs.
particle_for_check.dat	Фактически использованная после масштабирования частица внутри каталога результата.
FILE ИЗ --save-geometry	Effective native particle по явно выбранному стабильному пути.

Объект	СМЫСЛ
<result>_integrals.tsv	Machine-readable таблица основных extinction, scattering, absorption, efficiencies и оценки ошибки C_{sca} .

В лог и TSV выводится ровно по одному method-specific определению каждой физической величины:

$$C_{\text{ext}}, \quad C_{\text{sca}}, \quad C_{\text{abs}}, \quad Q_{\text{ext}}, \quad Q_{\text{sca}}, \quad Q_{\text{abs}}, \quad Q_x = \frac{C_x}{A_{\text{proj}}}.$$

В РО величина C_{ext} берется из optical theorem по forward amplitude. Для absorbing material $C_{\text{sca}} = \sum_j M_{11}(\theta_j) \Delta\Omega_j$ и $C_{\text{abs}} = C_{\text{ext}} - C_{\text{sca}}$. Для nonabsorbing material выполняется $C_{\text{sca}} = C_{\text{ext}}$ и $C_{\text{abs}} = 0$. В GO используется $C_{\text{ext}} = A_{\text{proj}}$; для absorbing case C_{sca} равен полной энергии выходных пучков, а C_{abs} — разности. В nonabsorbing GO снова принудительно выполняется физический баланс. Legacy hybrids и GO-величины в РО-отчете не печатаются; для GO, соответственно, не печатаются РО-величины.

Оценка точности интеграла C_{sca} сравнивает полную theta grid с nested grid из каждой второй строки:

$$\hat{\epsilon}_{\text{sca}} = \frac{|C_{\text{sca},h} - C_{\text{sca},2h}|}{\max(|C_{\text{sca},h}|, |C_{\text{sca},2h}|)}.$$

Для nonabsorbing calculation берется максимум этой оценки и независимой невязки conservation: интеграл M_{11} против optical theorem в РО или сумма выходных пучков против projected area в GO. При числе theta rows меньше пяти оценка недоступна. Значение больше 1 процента дает status check_csca_integration. Это a-posteriori проверка resolution, а не строгая верхняя граница ошибки и не проверка сходимости по orientations.

TSV содержит поля method, label, status, шесть значений C/Q и Csca_error_estimate_rel; альтернативных legacy определений в нем нет.

В начале и конце лога теперь записываются hostname, CPU model, logical и physical cores, effective OpenMP threads, total/available RAM, process RSS и peak RSS, видимые GPU и VRAM активного устройства. В конце также находятся полные FFT и cutoff reports. Поэтому время, измеренное при занятой другим процессом GPU, нельзя сравнивать с idle-device benchmark даже для того же binary.

Обычный запуск не создает диагностические файлы в текущем рабочем каталоге. Первый набор РО-пучков записывается только при явно заданном MBS_BEAM_LOG=PATH.

18. Практические правила валидации

1. Для сравнения CPU/GPU используйте `gpu/bin/mbs_po_gpu_double_fast --cpu` и тот же `binary` без `--cpu`. Так исключается различие CLI и большей части `host`-кода.
2. Для точек 0° и 180° используйте `--pole, double` и одинаковый `--grid`. Эти точки первыми показывают ошибки `endpoint/pole logic`.
3. Для `performance`-сравнения отдельно фиксируйте `backend`, точность и `environment`: `float_fast`, `double_fast`, `MBS_GPU_FUSED_MUELLER`, `MBS_GPU_STAGE_MUELLER` и число GPU.
4. Если менялись `Fresnel`, `RotateJones` или `edge-integral branches`, проверяйте не только M_{11} , но и M_{12} , M_{33} , M_{34} , M_{44} .
5. Если частица невыпуклая или агрегатная, отдельно проверяйте `runs` с `--trace_prefilter` и без него на малой сетке: результат должен совпадать в пределах `roundoff`.

19. Расширенные математические комментарии

19.1. Нормировка интенсивности

В коде амплитуда пучка и дифракционный интеграл разделены: трассировка дает J_{beam} , а РО-обработчик домножает ее на F_{edge} . Поэтому физически наблюдаемая интенсивность одной detector cell определяется не одним фактором, а произведением геометрической площади, фазового интеграла, Fresnel-множителей, оптического пути и нормировочного коэффициента output. В абстрактной записи:

$$I(\theta, \phi) \propto \left\| \sum_{b=1}^{N_b} C_{\text{norm}} F_{\text{edge},b} \mathbf{R}_{\text{out},b} \mathbf{F}_{n,b} \right\|_F^2.$$

Здесь $\|\cdot\|_F$ — Frobenius-норма. Важная практическая деталь: если сравнивать CPU и GPU, надо сравнивать не промежуточный F_{edge} отдельно, а весь pipeline, потому что часть фаз может быть вынесена в J_{phased} , часть в dirPhase , а часть в $\text{Fresnel/Jones matrices}$.

Таблица 21: Компоненты нормировки и где они могут проявляться.

Компонент	Практический смысл
C_{norm}	Общий множитель output; не должен зависеть от CPU/GPU backend.
F_{edge}	Дифракция апертуры; чувствительна к near-forward пределам.
$\mathbf{R}_{\text{out}}$	Поляризационный поворот; чувствителен к вырождению scattering basis.
$\mathbf{F}_n$	Накопленная Jones матрица и фаза optical path.
w_i	Вес ориентации; особенно важен при -pole.

19.2. Устойчивый предел edge-интеграла

Проблемный случай edge-интеграла возникает, когда

$$|A_q| \ll 1, \quad |B_q| \ll 1.$$

Тогда разность экспонент

$$\exp(i\Phi_{j+1}) - \exp(i\Phi_j)$$

вычитает почти равные числа. Устойчивая запись использует

$$\exp(iu) - \exp(iv) = \exp\left(i\frac{u+v}{2}\right) 2i \sin\left(\frac{u-v}{2}\right).$$

Если $\delta = (u - v)/2$ мал, то

$$\sin \delta \approx \delta - \frac{\delta^3}{6} + \frac{\delta^5}{120}.$$

Эта запись объясняет, почему forward direction должен обрабатываться отдельной веткой: обычная формула математически верна, но численно теряет значащие цифры. Для 0° и 180° такая потеря может выглядеть как различие CPU/GPU даже в double, если одна ветка попала в предел, а другая осталась в общей формуле.

19.3. Предел Jones-базиса на полюсах

Базис рассеяния зависит от

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\|\mathbf{v}_f \times \mathbf{s}\|}.$$

Если $\mathbf{s}$ параллелен $\mathbf{v}_f$, знаменатель равен нулю. Физически при точном forward/backward scattering азимутальный базис не единственен. Поэтому корректная численная процедура должна давать Mueller matrix, инвариантную к произвольному выбору азимута после соответствующего поворота Stokes-базиса:

$$\mathbf{M}_\chi = \mathbf{R}_M(\chi) \mathbf{M}_0 \mathbf{R}_M(-\chi).$$

Для случайно ориентированной частицы после интегрирования по ориентациям многие азимутальные зависимости сокращаются, но в одной фиксированной ориентации они могут быть видны. Поэтому для debug всегда надо отличать два вопроса: совпадает ли M_{11} как интенсивность и совпадают ли поляризационные элементы в одной и той же конвенции базиса.

19.4. Симметрия forward/backward Mueller

Для идеально случайной ориентации и полной азимутальной симметрии forward scattering имеет ограничения:

$$M_{12} = M_{21} = M_{13} = M_{31} = 0,$$

а блок Q/U становится диагонально-симметричным после выбора стандартного базиса. Backward scattering имеет собственные ограничения, но они зависят от конвенции Stokes parameters и от того, как реализован поворот в 180° . В коде есть функции ForwardScattering и BackwardScattering, которые явно зануляют часть элементов для специальных режимов. Если включен полный PO output, лучше проверять не только эти симметрии, но и непрерывность рядов около endpoint:

$$\lim_{\theta \rightarrow 0^+} M_{ij}(\theta) = M_{ij}(0), \quad \lim_{\theta \rightarrow \pi^-} M_{ij}(\theta) = M_{ij}(\pi).$$

19.5. Ошибка ориентационного усреднения

Для Monte Carlo ошибка типично имеет вид

$$\mathbb{E} \left[\left| \overline{M}_N - \overline{M} \right|^2 \right]^{1/2} \approx \frac{\sigma_M}{\sqrt{N}}.$$

Для low-discrepancy последовательностей Sobol/Hammersley в гладких задачах часто наблюдается более быстрое практическое падение:

$$\left| \overline{M}_N - \overline{M} \right| \lesssim C \frac{(\log N)^d}{N},$$

но это не строгая гарантия для всех частиц, потому что clipping, caustic-like features и endpoint branches делают интегранд негладким. Поэтому adaptive-критерий в коде сравнивает не теоретическую оценку, а реальные output rows.

19.6. Практическая формула выбора N_{ori}

Если известна грубая оценка относительной вариации V нужного элемента Mueller, то начальное число ориентаций можно брать как

$$N_{\text{ori},0} = \left\lceil \left(\frac{V}{\varepsilon_{\text{target}}} \right)^2 \right\rceil$$

для Monte Carlo-like режима. Для oldauto удобнее начинать с diffraction scale:

$$N_\beta \approx \frac{\beta_{\text{max}} - \beta_{\text{min}}}{\Delta\theta_{\text{diff}}/\text{DIV}}, \quad N_\gamma \approx \frac{\gamma_{\text{max}} - \gamma_{\text{min}}}{\Delta\theta_{\text{diff}}/\text{DIV}}.$$

Итого:

$$N_{\text{ori}} \approx N_\beta N_\gamma,$$

с поправками на симметрию и -pole. Если particle symmetry уменьшает gamma domain в s_γ раз, то

$$N_\gamma \leftarrow \left\lceil \frac{N_\gamma}{s_\gamma} \right\rceil.$$

19.7. Ошибка theta grid

Theta grid должна разрешать самый узкий физический feature. Для большого размера x forward peak имеет характерную ширину порядка

$$\delta\theta \sim \frac{1}{x}.$$

Если шаг $\Delta\theta$ слишком крупный, интегральная интенсивность может быть примерно правильной, но peak height и соседние rows будут неверными. Практическое правило:

$$\Delta\theta_{\text{front}} \leq \frac{1}{5x} \frac{180}{\pi} \quad \text{или строже для validation.}$$

Для backscattering обычно можно использовать более крупный шаг, но только если поляризационные элементы не имеют быстрых осцилляций. Поэтому --auto_tgrid пытается распределить rows неравномерно по физической сложности.

19.8. Сходимость по глубине трассировки

Глубина -n контролирует число внутренних событий. Если R — характерный Fresnel reflection magnitude, то вклад поколения g грубо падает как

$$I_g \sim R^{2g}$$

для непоглощающего случая без фокусировок. С поглощением добавляется множитель

$$\exp(-2k \operatorname{Im}(m) L_g).$$

Проверка сходимости по -n должна смотреть на финальный Mueller:

$$\varepsilon_n = \max_r \frac{\|\mathbf{M}_n(r) - \mathbf{M}_{n-1}(r)\|_\infty}{|M_{11,n}(r)| + \varepsilon_0}.$$

Если -n увеличить дорого, сначала включают beam cutoffs и анализируют число реально трассированных beams.

19.9. Beam cutoffs

Output cutoff удаляет уже готовый выходной пучок перед РО-дифракцией, а trace cutoff останавливает внутреннюю ветвь во время построения дерева отражений и преломлений. Три безразмерных критерия имеют вид

$$j_b = \frac{\|\mathbf{J}_b\|_F^2}{J_{\text{ref}}^2} < \varepsilon_J, \quad a_b = \frac{A_b}{A_{\text{ref}}} < \varepsilon_A, \quad i_b = j_b a_b < \varepsilon_I.$$

Здесь $\mathbf{J}_b$ — накопленная Jones-матрица ветви, а A_b — площадь beam polygon. Если включено несколько критериев, используется логическое OR: достаточно одного условия. Поэтому счетчики причин в логе могут перекрываться и их сумма не обязана равняться числу удаленных ветвей.

19.9.1. Единые профили

--cutoff-profile MODE нельзя смешивать с отдельными cutoff thresholds или --beam-area-ratio: конфликт завершается понятной ошибкой до начала расчета.

Таблица 22: Точные значения cutoff profiles.

Профиль	Output i_b	Trace i_b	Area ratio	Назначение
off	0	0	выключен	Reference, convergence и поиск ошибок.
safe	10^{-10}	10^{-12}	10^8	Первый production кандидат после сравнения с off.
balanced	10^{-8}	10^{-10}	10^5	Массовые scans после калибровки класса частиц.
fast	10^{-6}	10^{-8}	10^3	Предварительные расчеты, не validation.
legacy default	0	0	100	Совместимость, если профиль и отдельные пороги не указаны.

Профили включают combined importance, а не отдельные Jones/area OR-пороги. Так большая, но слабая ветвь или малая, но интенсивная ветвь не удаляется только по одному множителю. Важная деталь: без флагов сохраняется старое упрощение fragments с area ratio 100. Только --cutoff-profile off полностью отключает эту аппроксимацию. Профили не задают --trace-max-beams.

19.9.2. Отдельные пороги

Таблица 23: Индивидуальные cutoff controls.

Флаг	Диапазон	Действие
--beam-cutoff EPS	$[0, 1]$	Legacy shortcut: задает output Jones и area, удаление по OR.
--beam-cutoff-jones EPS	$[0, 1]$	Output $j_b < \text{EPS}$.
--beam-cutoff-area EPS	$[0, 1]$	Output $a_b < \text{EPS}$.
--beam-cutoff-importance EPS	$[0, 1]$	Output $i_b < \text{EPS}$.

Флаг	Диапазон	Действие
--trace-cutoff EPS	$[0, 1]$	Legacy shortcut: internal Jones и area, pruning по OR.
--trace-cutoff-jones EPS	$[0, 1]$	Internal $j_b < \text{EPS}$.
--trace-cutoff-area EPS	$[0, 1]$	Internal $a_b < \text{EPS}$.
--trace-cutoff-importance EPS	$[0, 1]$	Internal $i_b < \text{EPS}$.
--beam-area-ratio R	$R \geq 1$	После nonconvex split оставить только больший fragment, если отношение площадей не меньше R .
--trace-max-beams N	$N \geq 0$	Жесткий cap дерева на одну ориентацию; 0 отключает configured cap. Ненулевой hit означает truncation.

Общий legacy shortcut можно сочетать с соответствующим specific Jones/area значением: specific threshold заменяет свою сторону, а программа печатает warning. Для профиля такое частичное переопределение запрещено.

В конце расчета блоки OUTPUT BEAM CUTOFF REPORT и TRACE CUTOFF REPORT показывают effective thresholds, candidates, kept/rejected, причины, число упрощенных fragments, hits configured cap и hard tree limit. Калибровка должна сравнивать один и тот же набор ориентаций, n , theta и N_ϕ для off и выбранного профиля по всем 16 M_{ij}/M_{11} . Малый процент удаленных beams сам по себе не является оценкой ошибки из-за когерентной интерференции.

19.10. Double против float

В float машинное epsilon порядка

$$\varepsilon_{\text{float}} \approx 1.2 \cdot 10^{-7},$$

в double:

$$\varepsilon_{\text{double}} \approx 2.2 \cdot 10^{-16}.$$

Если суммируется N вкладов похожей величины, ошибка порядка сложения грубо может расти как

$$\varepsilon_{\text{sum}} \sim N \varepsilon_{\text{mach}}$$

для неблагоприятного порядка. Поэтому atomic float accumulation в GPU может отличаться от CPU сильнее, чем double. Для validation ветки, особенно около 0° и 180° , использовать double_fast; float_fast годится для production speed только после калибровки ошибки.

19.11. FFT/interpolation по phi

Пусть N_ϕ — число azimuth points в output, а f — compression factor. Первый CUDA pass напрямую считает примерно

$$N_{\phi, \text{direct}} = \max\left(16, \left\lfloor \frac{N_\phi}{f} \right\rfloor\right)$$

точек, затем periodic Fourier zero-padding восстанавливает полный output. Если $f = 1$, $N_\phi < 32$ или grid фактически не уменьшается, остается direct path без интерполяции.

--fft включает автоматический выбор f . Флаг --fft-factor N , $1 \leq N \leq 64$, явно задает степень сжатия и имеет приоритет над legacy environment variable MBS_FFT_PHI_FACTOR. Математически восстановление предполагает ограниченный spectrum:

$$f(\phi) = \sum_{m=-M}^M c_m e^{im\phi}.$$

Если реальный сигнал имеет существенные $|m| > M$, появляется aliasing или сглаживание; большое значение factor не является обещанием точности.

--fft-tolerance EPS, $0 < \text{EPS} < 1$, добавляет второй direct grid

$$N_{\phi, 2} = \min(N_\phi, 2N_{\phi, \text{direct}}).$$

Код сравнивает значимые M_{11} и все 16 элементов Mueller в масштабе M_{11} :

$$\hat{e} = \max\left(\max_{M_{11} \text{ significant}} \frac{|M_{11}^{(1)} - M_{11}^{(2)}|}{|M_{11}^{(2)}|}, \max_{i,j} \frac{|M_{ij}^{(1)} - M_{ij}^{(2)}|}{\max(|M_{11}^{(2)}|, 10^{-3}M_{11, \text{max}}^{(2)})}\right).$$

Если $\hat{e} > \text{EPS}$, сохраняется результат с удвоенным числом direct phi points. Это один a-posteriori refinement, а не строгая верхняя граница ошибки и не итерационная сходимость. Он требует дополнительного diffraction pass и одновременно хранит coarse, refined и output arrays, поэтому увеличивает время и peak memory.

```
# Explicit 4x compression; refine once above 1%
gpu/bin/mbs_po_gpu_float_fast ... --fft-factor 4 --fft-tolerance 0.01
```

Итоговый FFT INTERPOLATION REPORT содержит requested/effective factor, direct/output N_ϕ , число FFT-path calls, validation calls, refinements и наихудшую оценку. Factor 1 явно помечается как direct/no interpolation. Для reference следует убрать FFT-флаги или использовать --fft-factor 1; production factor сначала проверяют на характерных размерах и формах. Fused multi- k_{eq} FFT path не выполняет nested validation: при tolerance/check dispatcher переходит на поддерживаемый fallback, а не молча пропускает проверку.

19.12. Multi-size reuse

Для набора размеров геометрический beam tree может быть похожим, а фаза меняется через kD :

$$\Phi_j(k, D) = kD \xi_j,$$

где ξ_j — безразмерная геометрическая проекция вершины. Тогда можно переиспользовать нормированные вершины и slopes:

$$x_j(D) = D \hat{x}_j, \quad y_j(D) = D \hat{y}_j.$$

Это ускоряет scans по размеру, но требует осторожности: если размер меняет критерии cutoffs или auto grids, reuse может стать физически неэквивалентным полной пересборке.

19.13. Почему таблицы теперь не вылезают

В этой PDF-версии таблицы заданы как longtable с колонками фиксированной ширины:

$$p_1 + p_2 + \dots + p_n < W_{\text{text}}.$$

Длинный текст в колонках переносится с помощью RaggedRight, а не пытается держаться в одной строке. Поэтому таблицы с GPU flags, environment variables и длинными именами binaries остаются внутри A4. Для очень длинных inline names используется безопасный `\textbackslash detokenize`, чтобы underscore не превращался в математический индекс.

19.14. Протокол поиска расхождения CPU/GPU

Если найдено отличие между CPU и GPU, его надо локализовать сверху вниз:

1. Зафиксировать один binary и переключать только `--cpu`. Так исключаются разные CLI и host defaults.
2. Зафиксировать `double_fast`, `--grid`, `--fixed` или малый `--oldauto`, `-n`. Так уменьшается пространство причин.
3. Проверить `M11` отдельно от поляризационных элементов. Если отличается только `polarization block`, вероятнее проблема в `basis rotation`.
4. Проверить `--pole` и без `--pole`. Если отличие только на `endpoint`, причина в `beta/gamma weight` или `scattering basis limit`.
5. Отключить `fused/staged paths environment variables` и сравнить `atomic/no-atomic path`. Если меняется только последний разряд, это `reduction order`; если меняются проценты, это ветка формулы.

6. Сравнить одну ориентацию `--fixed`. Если `fixed` совпадает, а `average` нет, причина в `orientation weights/chunks`.

19.15. Протокол выбора production-параметров

Практический production setup:

1. На малом размере подобрать `-n`, `cutoffs` и `theta grid`.
2. На среднем размере сравнить `--oldauto 2`, `--oldauto 4`; если разница велика, использовать 4 или `adaptive`.
3. Проверить `float_fast` против `double_fast` на нескольких `representative sizes`.
4. Для финального scan включить `multi-GPU` или `multi-size reuse` только после проверки, что один размер совпадает с `non-reuse` расчетом.
5. Сохранить `command line`, `git commit`, `CUDA driver`, `GPU model`, `compiler flags` и `environment variables` вместе с `output`.

19.16. Формат частицы из файла

Точный формат `--pf` зависит от используемого reader в репозитории, но математически любая `file particle` сводится к набору вершин и граней:

$$\mathcal{P} = \left(\{\mathbf{v}_i\}_{i=1}^{N_v}, \{\mathcal{F}_j\}_{j=1}^{N_f} \right).$$

Грань $\mathcal{F}_j$ является упорядоченным списком индексов вершин:

$$\mathcal{F}_j = (i_{j,1}, i_{j,2}, \dots, i_{j,m_j}).$$

Нормаль грани вычисляется из обхода вершин. Для треугольной части:

$$\mathbf{n}_j = \frac{(\mathbf{v}_{i_2} - \mathbf{v}_{i_1}) \times (\mathbf{v}_{i_3} - \mathbf{v}_{i_1})}{\|(\mathbf{v}_{i_2} - \mathbf{v}_{i_1}) \times (\mathbf{v}_{i_3} - \mathbf{v}_{i_1})\|}.$$

Если обход задан в обратном направлении, нормаль меняет знак. Это критично: Fresnel ветвление, `inside/outside test` и `clipping` используют ориентацию нормали. Для невыпуклой частицы дополнительно важно, чтобы грани не были самопересекающимися и чтобы соседние полигоны имели согласованную топологию.

Таблица 24: Проверки file particle перед расчетом.

Проверка	Зачем нужна
Нет нулевых ребер	Иначе нормаль и edge slopes становятся неопределенными.
Грань почти плоская	РО-апертура предполагает плоский polygon.
Нет самопересечения грани	Polygon clipping и area formula требуют простого полигона.
Согласованный обход	Нужен правильный знак нормали.
Нет дублированных почти совпадающих граней	Иначе tracing может давать двойные hits и unstable clipping.
Разумный масштаб	Слишком маленькие или большие координаты ухудшают conditioning.

19.17. Площадь, объем и центр

Площадь полигона в локальной плоскости:

$$A_j = \frac{1}{2} \left| \sum_{l=1}^{m_j} (x_l y_{l+1} - x_{l+1} y_l) \right|.$$

Центр грани:

$$\mathbf{c}_j = \frac{1}{m_j} \sum_{l=1}^{m_j} \mathbf{v}_{i_j, l}.$$

Объем замкнутого многогранника удобно выражается через тетраэдры с началом координат:

$$V = \frac{1}{6} \sum_{\text{triangles}} \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}).$$

Знак объема снова зависит от ориентации обхода. Если volume получается отрицательным или близким к нулю для явно объемной частицы, это почти всегда проблема геометрии.

19.18. Разбор строки output

Типовая строка output содержит угол, вес и 16 элементов Mueller:

$$\theta_j, \quad \Delta\Omega_j, \quad M_{11}, M_{12}, \dots, M_{44}.$$

В терминах кода:

$$M_{11} \equiv M_{00}, \quad M_{44} \equiv M_{33}.$$

Если output усреднен по ориентациям, то каждый элемент уже содержит сумму:

$$M_{ab}^{\text{out}}(\theta_j) = \frac{1}{W} \sum_i w_i M_{ab}^{(i)}(\theta_j).$$

Если затем нужна интегральная характеристика по углам, используется вес $2\pi \cdot d\cos$:

$$\langle M_{ab} \rangle_{\Omega} = \sum_j M_{ab}^{\text{out}}(\theta_j) \Delta\Omega_j.$$

Не надо повторно умножать на 2π , если колонка уже называется $2\pi \cdot d\cos$.

19.19. Матрица validation-тестов

Таблица 25: Минимальная матрица проверок после изменения физического кода.

Тест	Команда/режим	Что должен поймать
Single fixed orientation	--fixed B G --grid 0 180 ...	Ошибки RotateJones, Fresnel, edge integral.
Forward/back only	--grid 0 180 N 1 --pole	Endpoint, pole weight, singular basis.
CPU from GPU binary	--cpu vs default GPU	Различия backend при одинаковом host code.
Double vs float	double_fast vs float_fast	Потери точности и fast math.
Incoherent check	--incoh	Ошибки порядка Jones sum vs Mueller sum.
Non-convex clipping	--trace_prefilter on/off	Ошибки projected-AABB rejection.
Orientation convergence	--oldauto 2/4/8	Недостаточное число ориентаций.
Depth convergence	-n N and -n N+1	Недостаточная глубина трассировки.

19.20. Типичные причины больших расхождений

Таблица 26: Диагностика расхождений.

Симптом	Вероятная причина	Что проверить
Отличается только 0°	Endpoint/pole branch	--pole, theta cell boundary, basis limit.
Отличается только 180°	Backward basis/Fresnel sign	--legacy_sign, BackwardScattering, RotateMueller.
M_{11} совпадает, M_{33}/M_{34} нет	Polarization basis convention	RotateJones VS KarczewskiJones.
Float сильно отличается от double	Cancellation или fast math	Near-forward rows, atomic reduction, --use_fast_math.
GPU отличается только при large grid	Reduction order или chunking	fused/staged env vars, chunk size.
Non-convex результат нестабилен	Clipping candidate error	prefilter on/off, facet normals, self-intersections.
Auto дает другой результат, чем oldauto	Разные theta/orientation grids	сохранить auto grid и повторить вручную.

19.21. Контроль conservation-like величин

Для непоглощающих частиц полезно смотреть на интегральные величины, хотя РО+ГО приближение не обязано идеально выполнять все теоремы для любой численной сетки. Если $S_{11}(\theta)$ соответствует M_{11} , интегральная scattering-like величина:

$$C_{\text{sca,num}} = \sum_j S_{11}(\theta_j) \Delta\Omega_j.$$

При увеличении theta resolution и orientation resolution эта величина должна стабилизироваться. Если $C_{\text{sca,num}}$ скачет при небольшом изменении NTH, это признак недоразрешенного forward peak.

19.22. Минимальные данные для воспроизводимого отчета

Вместе с output надо сохранять:

Таблица 27: Reproducibility metadata.

Данные	Почему нужны
Git commit	Физика, CLI и CUDA kernels могли измениться.
Полная command line	Defaults могут отличаться между версиями.
Compiler и ARCH_FLAGS	AVX path и floating-point contraction зависят от сборки.
CUDA toolkit и driver	Разные версии меняют libdevice, cuFFT и scheduling.
GPU model	Разная скорость double/float и atomics.
Environment variables	GPU fused/staged/multi режимы выбираются через env.
Particle file hash	Геометрия file particle должна быть точно той же.

19.23. English technical summary

MBS-fast is a ray-tracing plus Physical Optics solver for faceted nonspherical particles. The ray-tracing stage constructs beam polygons and propagates complex Jones matrices through Fresnel reflection/refraction events. The PO stage evaluates a polygon-aperture diffraction integral in edge form. For one orientation, the default mode sums Jones amplitudes coherently and converts the resulting 2×2 complex matrix into a real 4×4 Mueller matrix. Random-orientation output is a weighted average over beta/gamma grids, Sobol/Hammersley/lattice sets, or adaptive orientation sequences.

The most delicate numerical locations are forward and backward scattering. At these endpoints the scattering basis can become singular and the edge integral reaches a small-denominator limit. Correct CPU/GPU agreement therefore requires the same endpoint logic, the same pole weights for orientation averaging, and the same phase/Fresnel convention. Production speed on GPU comes from parallelizing beam-direction-orientation work items and from fused kernels that avoid writing large intermediate Jones buffers. CPU speed comes from OpenMP/MPI decomposition and AVX-512/AVX2 packing of phase and trigonometric evaluations.

А. Краткая английская шпаргалка

This PDF is the authoritative Russian manual. The key English mapping is:

Russian term	English term
трассировка лучей	ray tracing
пучок	beam
грань	facet
апертура	aperture
коэффициенты Френеля	Fresnel coefficients
матрица Jones	Jones matrix
матрица Mueller	Mueller matrix
дифракционный edge-интеграл	diffraction edge integral
усреднение ориентаций	orientation averaging
невыпуклый clipping	non-convex clipping
агрегат	aggregate

В. Связь с файлами кода

Таблица 29: Где искать описанные объекты в коде.

Файл/область	Что там находится
src/main.cpp	CLI flags, auto/oldauto/adaptive orchestration, help text.
src/Splitting.cpp	Fresnel coefficients, reflected/refracted beam directions, optical path update.
src/Beam.* src/scattering	Beam geometry, Jones matrix, polygon state. Ray/facet intersection, convex and non-convex clipping.
src/handler/Handler.cpp src/handler/HandlerP0.cpp	Scalar and fast edge diffraction integrals. Jones rotation, ApplyDiffraction, Mueller accumulation, AVX paths.
src/math/Mueller.cpp src/cuda ИЛИ *.cu	Jones-to-Mueller and Mueller rotations. CUDA diffraction kernels, atomic/no-atomic/fused paths.

С. Приложение: контрольные формулы для отчетов

С.1. Относительная ошибка элемента Mueller

Для сравнения двух расчетов A и B удобно печатать относительную ошибку по каждому элементу:

$$\delta_{ij}(\theta) = \frac{M_{ij}^A(\theta) - M_{ij}^B(\theta)}{|M_{11}^A(\theta)| + \varepsilon_0}.$$

Нормировка на M_{11} полезна, потому что слабые поляризационные элементы могут проходить через ноль. Для абсолютного контроля слабых элементов дополнительно используют

$$\Delta_{ij}(\theta) = M_{ij}^A(\theta) - M_{ij}^B(\theta).$$

В отчетах лучше указывать обе величины: относительная ошибка показывает физический масштаб, абсолютная ошибка показывает численный шум около нулей.

С.2. Интегральная ошибка по углам

Если сравниваются полные кривые, вводится weighted RMS:

$$E_{ij}^2 = \frac{\sum_j (M_{ij}^A(\theta_j) - M_{ij}^B(\theta_j))^2 \Delta\Omega_j}{\sum_j (|M_{11}^A(\theta_j)| + \varepsilon_0)^2 \Delta\Omega_j}.$$

Такая метрика не заваливается на одной точке, но endpoint 0° и 180° все равно надо проверять отдельно, потому что именно там находятся специальные численные ветки.

С.3. Сравнение времени

Время расчета надо раскладывать на компоненты:

$$T_{\text{total}} = T_{\text{trace}} + T_{\text{diffraction}} + T_{\text{reduction}} + T_{\text{io}}.$$

GPU ускоряет в основном $T_{\text{diffraction}}$ и часть $T_{\text{reduction}}$. Если частица невыпуклая и clipping дорогой, T_{trace} может доминировать, и GPU speedup будет ниже. Для честного сравнения CPU/GPU надо фиксировать:

$$N_{\text{ori}}, \quad N_\theta, \quad N_\phi, \quad N_b, \quad n, \quad \text{cutoffs}.$$

С.4. Оценка памяти GPU

Грубая оценка памяти для Jones buffer:

$$B_J \approx N_\theta N_\phi N_{\text{ori,chunk}} \cdot 4 \cdot 2 \cdot s_{\text{real}},$$

где 4 — число комплексных элементов Jones, 2 — real/imag, $s_{\text{real}} = 4$ для float и 8 для double. Для Mueller buffer:

$$B_M \approx N_\theta N_\phi N_{\text{ori,chunk}} \cdot 16 \cdot s_{\text{real}}.$$

Fused kernels уменьшают потребность в полном B_J , потому что Jones сумма может жить в регистрах или shared/local memory до преобразования в Mueller.

D. Приложение: чеклист перед публикацией чисел

1. Указать ветку и commit.
2. Указать CPU/GPU binary, точность и fast-math вариант.
3. Указать particle source: built-in параметры или hash file particle.
4. Указать λ , $m = n + i\kappa$, размер, -n.
5. Указать orientation method и число ориентаций.
6. Указать theta/phi grid и наличие --pole.
7. Указать cutoffs и clipping options.
8. Указать environment variables GPU.
9. Приложить сравнение хотя бы двух grid/orientation resolutions.
10. Для GPU production приложить контроль double на малой подвыборке.

D.1. Минимальная таблица в статье или отчете

Таблица 30: Поля, которые должны быть в reproducibility table.

Поле	Пример
Code	MBS-fast main, commit abcdef0.
Backend	gpu/bin/mbs_po_gpu_double_fast.
Particle	-p 1 125.9 78.09 ИЛИ --pf particle.dat.
Optics	--ri 1.3116 0 -w 0.532 -n 12.
Orientations	--oldauto 2 --pole.
Scattering grid	--grid 0 180 600 180 --close.
Hardware	CPU model, GPU model, driver, CUDA toolkit.
Validation	CPU/GPU max error, oldauto 2/4 error, depth error.

D.2. Что не надо смешивать в одном сравнении

Нельзя одновременно менять backend, точность, orientation grid и theta grid, а затем приписывать отличие GPU. Корректная серия меняет один фактор:

$$A_0 \rightarrow A_1 \rightarrow A_2 \rightarrow \dots,$$

где каждая стрелка меняет только один параметр. Например:

CPU double $\rightarrow$ GPU double $\rightarrow$ GPU float $\rightarrow$ GPU float fast.

Тогда видно, на каком шаге появляется расхождение.

E. Appendix: English flag groups

Таблица 31: English grouping of the most important flags.

Group	Flags
Method	--po, --go, --incoh.
Particle	-p, --pf, --rs, --k_eq, --multigrid, --multikeq, --multikeq_list.
Optics	--ri, -w, -n, --abs, --legacy_sign.
Orientation	--fixed, --oldauto, --sobol, --adaptive, --auto, --autofull, --pole.
Scattering grid	--grid, --tgrid, --auto_tgrid, --auto_phi, --nphi.
Backend	--gpu, --cpu, --threads.
Diagnostics	--jones, --save_betas, --noshadow_output, --karczewski, --help-debug.

The physically safest validation run is double precision with a direct grid, a fixed small set of orientations, no FFT interpolation, and the same binary used for CPU and GPU by toggling --cpu. The fastest production run is usually GPU float or GPU float-fast with fused Mueller kernels and chunked orientation batches, but that mode must be calibrated against double precision for the particle class and size range under study.